

The Ship of Theseus — v0.25

One document, three parts. Part I is the specification: what Theseus is meant to be. Part II is the build plan: the order it is built in, with the test that gates each step. Part III is the record of what was actually built, milestone by milestone, and where it diverged from Parts I and II. The document is therefore both spec and documentation; when the code and Part I disagree, Part III says so and one of them gets fixed.

*Consistency pass by Tabitha, 2026-09-24, folding three deep dives (sessions, shells, memory) and all of Eddie's answers into one coherent document. Earlier provisional text that the deep dives superseded has been removed rather than annotated. **[D]** marks a provisional decision Tabitha made to keep the document whole; overturn freely. v0.5 incorporated an external review (GPT Astra, 2026-09-24; Appendix A). v0.6 closed the last open question. v0.7 added the hooks surface (§3.17). v0.8 adopted the event-driven execution model from Eddie's all-webhook proposal as reviewed in `notes/event-design-review.md` (§3.3, §3.16, §6, §7, §8). v0.9 incorporated the second external review (Appendix D) and renamed the document at Eddie's request. v0.10 records Eddie's decisions on the three questions it left open (storage kernel, shell default, control-plane separation) and adds the turn-lock durability model. v0.11 answers the concurrency question (what runs "simultaneously": one turn per **session**, where a session is a task or a conversation, §3.2a) and folded the build plan into this document as Part II. v0.12 settles when a session is recompiled (§4.4a): append by default, recompile only on need, with Jev owning the judgment; estimates are removed from Part II; the repository is `github.com/zeroaltitude/theseus`. v0.13 states how sessions persist across runtime restarts as lineages in a multi-parent graph (§4.4b). v0.14 defines **loop**, **turn**, and the **Advancer** (§3.3a), the wire protocol and client isolation (§3.18), the secrets posture (§3.19), and replaces the first milestone with **M0 First light**, a vertical slice Eddie specified. v0.15 added Part III (as built) and the standing rule that actuals are recorded as work lands (Eddie, 2026-09-25).*

Ariadne held the thread. Theseus walks the labyrinth on it.

Part I — Specification

0. Thesis

Theseus is a **durable agent runtime with a provenance-aware context compiler**: work has an identity, authority has a boundary, actions have recoverable histories, and context is compiled from one logical history with a manifest, appended to by default and recompiled only when something real demands it. Concretely, it is a single statically linked Rust binary that runs thousands of concurrent conversations, talks to humans through Discord in text and voice, acts on the world through AWS-native tools and graded shells, and owns its own turn loop against the Anthropic Messages API. The loop does not stop on a counter; it stops when Jev judges that the work has reached a definable

stopping point, and otherwise keeps going. Context is one multi-rooted, typed-edge graph held in memory, from which every prompt is assembled fresh. Tasks, memory, judgments, and self-tuning are intrinsic to the runtime. Every capability is a plugin against a small core, but the shipped binary is opinionated: Discord, AWS, Anthropic, and Jev are always present, and MCP is a first-class capability in both client and server roles.

It runs on one large node. That node may be an EC2 instance or Eddie's desktop. "AWS native" means the AWS tool surface is deep and default, not that the harness must live in AWS.

1. Settled decisions

Topic	Decision
Language, artifact	Rust. One statically linked binary per target. No dynamic linking, no sidecars; helper processes are the same binary launched with a role flag.
Licence	Open source, dual-licensed MIT OR Apache-2.0 in the Rust convention (decided 2026-09-25): Apache's patent grant and contribution clause for those who want them, MIT's brevity for those who do not. Permissive-only dependencies enforced by <code>cargo deny</code> from the first commit. No AGPL (rules out linking Vestige).
Comms	Discord only, text and voice. One Discord application invited to many guilds.
Model path	Direct Anthropic Messages API. Bedrock is a possible later provider, not the default.
Hands	AWS tool surface is deep and default. Shells are graded: local host, local native sandbox, and AWS classes, chosen per job by Jev within policy. L0 (native host shell) is the default for BigHat's deployment; agent-authored code and package installs go to L1; the open-source distribution ships L1 as default with L0 as documented opt-in. Committed 2026-09-24, to be revisited on evidence.
Execution model	Event-driven. No in-flight state lives only in harness memory; every dispatched thing is a WAL record with a harness-minted correlation id; completion arrives as an event (in-process, Unix socket spool, SQS pull, loopback HTTP as the off-by-default exception); the harness is quiescent between events; the one-minute heartbeat is the level-triggered reconciler. Adopted 2026-09-24 from Eddie's all-webhook proposal, with "webhook" generalized to "completion event" and "unkillable" replaced by "detached, durable, cancellable" (§3.3, §3.16).
Deployment	One large node: Theseus, source trees, and sandboxes together. Must also run on a home desktop with every AWS dependency optional at runtime.
Durability	Local fsync to a persistent SSD is the floor, before any action is dispatched. Off-node durability is eventual, 5–60 s (measured, not asserted), produced by asynchronous durability work the core performs in the time a turn has surrendered to a remote (model call, shell, judge, human). Single node; no replication.
Storage kernel	Decided by measurement (M1, 2026-09-26): the WAL is the truth; a pure-Rust embedded store is a rebuildable index over it, and that store is <code>redb</code>. On identical fsync-bound workloads the two candidates append at the same rate (the disk decides); with fsync removed <code>fjall</code> appends 1.7× faster but <code>redb</code> reopens 2× faster and answers "latest of every key" 10–17× faster, is one file, and has no background compaction. <code>fjall</code> remains behind the <code>Index</code> trait, selectable by config; a store keeps the engine it was created with.

Topic	Decision
Unit of concurrency	One turn per session. A session is a task or a conversation; each has exactly one execution and one turn lock. Thousands of sessions exist durably; those with runnable work run concurrently, bounded by an admission scheduler; the rest are parked at zero cost. Channels order deliveries, not work. (Eddie, 2026-09-25.)
Graph shape	The context graph is a DAG with multiple parents , not a tree. A node belongs to a channel, to a session, to any number of compilations, and to derived nodes at once, each by its own typed edge. Order within any lineage comes from the node's WAL position, never from a single <code>next</code> chain.
Session persistence	A session persists as a small record pointing at its current <code>Compilation</code> and its tail range; it is durable by reference and resolves to a lineage of compilations linked by <code>derived_from</code> . Nothing is copied per session; rendered prefixes are a rebuildable cache.
Secrets	All secrets live in 1Password , in the deployment's vault, read at startup through a service account. The only secret the process may receive any other way is the service-account token itself. Config references secrets as <code>op://vault/item/field</code> ; no secret value is ever written to disk, config, log, or ledger.
Wire protocol	The core is a server speaking JSON-RPC 2.0 over newline-delimited JSON , on stdio when spawned and on a Unix domain socket as a daemon. Every client, including the in-binary CLI and later Discord and the web UI, talks to the core only through this protocol.
Loop, turn	A loop is one pass through the toolchain manager to the provider and back. A turn is the sequence of loops run under one acquisition of a session's turn lock, ended by the Advancer.
Recompilation	A session's compiled context is appended to by default and recompiled only on need . Deterministic triggers (audience, policy, schema, window) force it; otherwise Jev judges whether a real-world change warrants it. Long single threads therefore evolve exactly like a plain transcript, and cache their prefix. (Eddie, 2026-09-25.)
Repository	<code>~/projects/theseus</code> , <code>github.com/zeroaltitude/theseus</code> .
Turn lock	Per execution, exactly one turn advances at a time (the "GIL"). The lock is held only during local work and released at every offload boundary; the freed time is spent on durability, indexing, and maintenance.
Shell default	L0 and L1 both ship in the first useful agent. L0 is the early operator default, explicitly subject to change once L1 has real mileage.
Control-plane separation	Running the runtime and its storage under a separate OS identity from L0 jobs is an option, strongly recommended in the documentation and the installer, not a default.
Agents	One agent, many roles. No multi-agent, no inter-agent conversations.
Conversations	A conversation is the subset of the node graph that happened on a particular channel, in temporal order. Nothing more: it is a derived view, not a stored object, and it never ends because a channel never stops accumulating nodes. Many participants. Each execution (§3.15) acts in exactly one channel at a time; many executions run concurrently across channels.
Tasks	Fluid and conversational. The agent sees the whole task graph and reshapes it with the operator or alone.
Memory	Not a separate store. The graph is append-only, always : compaction <i>adds</i> summary nodes and later views may not include what was trimmed, but no node is ever lost. "Memory" is any node that retains enough strength to be selected into a prompt; decay lowers selection weight and moves payload to cold storage, it never removes anything. Implemented natively behind a <code>MemoryScience</code> trait; Vestige not used.

Topic	Decision
MCP	Full mode: Theseus is both client and server. Prompts and elicitation in; sampling in, budgeted and Jev-judged.
Proactivity	The agent may open a conversation with a human unprompted. Safety rests on the Jev security classifier plus a default-safe operator environment.
Destructive confirm	Goes to the person who issued the request, as a Discord component. When there is no requester (proactive or scheduled work), it goes to the owner . Timeout means no action.
Owner vs operators	One owner per deployment: whoever runs the Theseus runtime, whether that is Eddie, a company's CTO, or a single person on their own laptop. The owner holds final authority over policy, budgets, and unrequested destructive actions. Many operators may converse with and configure the system within what the owner allows.
Unprompted actions	All allowed without confirm: DM a human, open a thread, post in a channel, speak in voice. The agent must be invited to a channel first, voice or text; it never joins uninvited.
Task terminal/stalled states	Judged by Jev like every other loop state; escalated to a human only when Jev's confidence is in question. No separate verification tier. Deterministic control paths (/stop , revocation, budget exhaustion) bypass Jev entirely (§3.15).
Provider outage	Fail closed and say so in Discord. No secondary provider.
Voice mode	PTT vs VAD is a human Discord preference, not an agent concern.
MCP server auth	Deferred; a single static API key for now.
Budgets	A first-class, flexible notion attachable to parts of the system by design; semantics deliberately unspecified for now.
Reachability	Localhost only, outbound OK, no inbound. External proxies handle exposure.
Encryption at rest	Required, provided by the platform (EBS volume encryption, LUKS on desktop), not by Theseus.
Roles	Hints to the model, never enforced policy. Policy lives in §3.9 only. Upgradable later if it fails.
Redaction	Append-only is the logical history model. Payload erasure is an allowed, receipted exception for secrets and other must-not-exist content, with lineage-aware invalidation (§5.6). Suppression alone is not sufficient.
Deferred	GDPR/right-to-be-forgotten, MCP token scopes (one static key = one shared principal under the binding ceiling, accepted for now). Backup <i>restore path</i> is not deferred (§6); backup <i>drills</i> remain deferred.
Policy administration	The operator administers the Discord-role-to-policy mapping.
Observability	CloudWatch for historical search. An in-binary web UI, in the spirit of the OpenClaw gateway UI, for immediate and local: conversation snooping, RL feedback, category management and scoring nudges.
Embeddings	Local Nomic Embed v1.5 in the index tender, shared by all nodes, model id stamped on every vector.
Name	Theseus.

2. Principles as constraints

Principle	Constraint it imposes
EFFICIENT	An idle conversation costs kilobytes. Ten thousand on one node is the target; hundreds already beats everything but a hyperscaler's managed agents at a fraction of the cost.
TASKS	A persisted task graph, read whole every turn, edited through structured actions the harness executes. No MCP, no free-text tool for tasks.
MEMORY	One graph. Jev labels kind, durability, and trust. Every node eligible every turn; the indexer and budgeter make that cheap.
LOOP FOREVER	Default is to continue while authorized, runnable work exists within reserved resources . Stopping requires a positive, logged reason: a Jev judgment or a deterministic control path. Budget exhaustion is correct operation, never a judge failure, even when the last judgment said <code>progressing</code> : a task can be progressing honestly when its allotment runs out. The ledger records <code>budget_exhausted</code> as its own outcome class so the stopping policy is never trained against honest progress.
OPINIONATED	One blessed path, few knobs, strong defaults. Discord's model is the session model. AWS is the tool model.
JEV IN THE LOOP	Continue/stop, role, continuation strategy, shell class, memory labels, security risk, and MCP sampling proportionality are all typed judgments over bounded state.
LEARNING	Every judgment is recorded with inputs, action, and later outcome. Question packs, memory-science parameters, and shell mappings are versioned data tuned by that record. Precisely: feedback-driven policy and parameter optimization with holdouts and canaries (§3.10), not reinforcement learning in the technical sense.
SPEED, DURABILITY, COST	In that order, for storage and for every runtime trade.
QUIET BY CONSTRUCTION	The harness has no busy loop. It parks on its event sources and the heartbeat. Work in flight is a durable record, not a waiting thread.
NATIVE FIRST	The default way Theseus does anything is a small, typed, in-process Rust toollet. Shelling out is the escape hatch, ledgered as such; the ratio of shell calls to native calls is a health metric, and the most frequent shell patterns are the queue for the next toollet. Typed arguments are what let policy read intent, provenance ride on outputs, and the learning loop see what the agent actually does. (Eddie, 2026-09-26.)
EXQUISITE VISIBILITY	Every turn, loop, provider call, hook site, judgment, and completion is timed and attributed as it happens, in the record, before anyone asks. Statistics and visualization are built with the feature, not after it. Nothing that matters is sampled away, and any OpenTelemetry backend can be pointed at the running system for service-level stats without code changes. We move carefully because we can see. (Eddie, 2026-09-25.)
APPEND-ONLY	The event record only grows. Compaction, supersession, suppression, and forgetting are new records; nothing in the record is rewritten. Projections (retention, heat, task state, trust annotations, indexes) are mutable and rebuildable from the record. Payload erasure under §5.6 is the single, receipted exception.

3. Architecture



3.1 Bindings (channel configuration)

Configuration is a list of bindings, resolved most-specific-first (thread over channel over guild). Each selects a Discord scope and attaches behaviour:

```
binding:
  scope: { guild, channel | thread | dm | voice_channel, optional user/role filter }
  persona: system prompt, name, default voice
  roles: allowed role set and default weights (see §3.4)
  policy: tool tags allowed, confirm thresholds, spend ceilings, shell classes allowed, p
  model: provider/model, thinking effort, optional Jev cheap-first cascade
  memory: namespaces read and written
  loop: judge pack versions, ceilings
  mcp: attached servers, which may sample, which may elicit
  listen_only: bool # ingest to memory, never reply
```

Bindings live in a versioned config in the store, editable by slash command and by the web UI with an audit trail. The Discord-role-to-policy mapping is a top-level table the operator administers through the same two surfaces.

3.2 Channels and conversations

Channel is a Discord place under a binding: a thread, channel, DM, or voice channel. Channels are cheap, unbounded, and carry presence, permissions, and Discord message ids. Channel is also a classification dimension: where something is happening, and where past things happened.

Conversation has a deliberately tight definition: **the subset of the node graph that happened on a particular channel, in temporal order**. It is a view derived from the graph, not a stored entity with its own lifecycle. Its identity *is* the channel. It never ends because a channel never stops accumulating nodes; it can only be continued, re-rooted by compaction, or left dormant. The `next` chain is therefore per channel, and `in_conversation` is not a separate edge type; `in_channel` plus temporal order defines it.

Consequences:

- Many participants, each a `Person` node with `by` edges; no per-human forks.
- A voice channel and its paired text channel under one binding are two channels and therefore two conversations that share participants, tasks, and memory scope; the agent borrows across them freely (below), which is what makes them feel like one.
- Each execution (§3.15) reports into exactly one channel at a time; many executions run concurrently. A channel orders what is *delivered* to it, never what is *computed* for it (§3.2a). "Gliding" is an execution moving between channels, and pulling other channels' history into the current one.
- A reference to another channel's history ("what we did in the deploy thread") makes the agent pick it up: **borrow** by default (that channel's summary or relevant nodes enter the current context via `mentions_conversation`), **switch** on explicit ask (subsequent turns are posted and appended in that channel instead).

The runtime object behind a channel is a `tokio` actor with a mailbox and a few kilobytes of hot state. Parked, it costs no CPU and no thread. Turns are serialized per channel; messages arriving mid-turn are coalesced into the next context build **with each message's author preserved**, so authority never blurs across a coalesced batch. Under LOOP FOREVER, an inbound message during autonomous work is a nudge, not an interrupt, unless it is a deterministic control (`/stop`, `/cancel`, revocation) or Jev judges it a new ask.

Humans are `Person` nodes keyed by Discord user id with their own memory namespace that follows them across every guild and channel.

3.2a Sessions: what runs "simultaneously"

The word *session* is used deliberately and narrowly. A **session is a compiler scope**: the thing that decides which roots the context compiler starts from, which audience it compiles for, which budget it spends, and which continuation strategy it uses. It is not a store and not a transcript. There are exactly two kinds:

- A **conversation session**, one per channel: root is the channel's temporal view; audience is the channel's participants; cadence is human (a reply is expected soon).
- A **task session**, one per task that has been *promoted* to autonomous work: root is the `Task` node, with its evidence, its subtasks, and the conversation that created it as secondary roots; audience is whoever it reports to; cadence is autonomous (no one is waiting on the next token).

One turn per session. Every session has exactly one execution (§3.15) and that execution has one turn lock. "Hundreds or thousands of sessions running simultaneously" therefore means: thousands of task and conversation sessions exist as durable nodes; at any instant those with runnable model work hold a turn, the rest are `waiting` and cost nothing. An **admission scheduler** bounds how many hold a turn at once by budget, provider rate limits, and a configured concurrency ceiling; it is a queue, not a policy, and it never reorders deterministic control paths (`/stop`, `/cancel`, revocation).

Promotion. A conversation accumulates intent; at some point it becomes a structured, trackable, goal-oriented piece of work. That moment is `task.create` with `autonomous: true` (proposed by the model, judged by Jev under `CLASSIFY`, or asked for by a human). Promotion **forks an execution**: the new task execution inherits the requesting principal's authority and delegation limits (never broader, §3.9), receives its own budget allotment carved from the requester's, records `origin_channel`, and gets a `reports_to` edge to that channel. The conversation session returns to its human cadence immediately; it is not blocked by the task and never was the task.

How they stay connected. Only through the graph, never through shared in-memory state:

- The task execution writes `Progress`, `Question`, and `Result` nodes with `reports_to` edges. Delivery to the channel is an *action* (§3.16), ordered by the channel like any other post. The channel never waits for the task to compute; it only orders what the task says.
- The conversation session's compiler admits a bounded summary of the channel's live task executions (state, last progress, open questions) every turn, so the agent talking to humans always knows what its background hands are doing.
- A human message in the channel is classified: a nudge or new ask for the conversation; a **control** (`/stop`, `/cancel <task>`); or **addressed to a task**, in which case it is written as a `Message` node with an `addressed_to` edge and *wakes* the task execution, whose next turn coalesces it with the author preserved. Ambiguity resolves toward the conversation, which can always hand it on.
- A task that needs a human (a confirm, a question, a blocked gate) enters `waiting` with a `Question` node delivered to its channel and its requester; the answer wakes it. This and terminal states (`complete`, `failed`, `budget_exhausted`) are the only points at which a task is synchronous with anyone.
- A task session may **glide** like a conversation (report into a different channel; borrow another channel's history), under the intersected-ceiling rule.

Sub-tasks. By default a task's subtasks run inside its one execution, sequentially, under its one turn lock; the task is the unit of autonomy, not the subtask. A subtask is promoted to its own session only by an explicit `task.create {autonomous: true}` from inside the task, bounded by the parent's remaining budget and a per-task fan-out cap; the child's `reports_to` is the parent task, and its terminal state is a `Completion` for the parent. Fan-out is therefore a tree of sessions with budgets that sum, never a swarm.

Resources. Sessions do not serialize resources. Two task sessions can touch the same checkout or the same task node; §3.5's CAS versions, claim leases, and workspace locks are what arbitrate, and a lost lease is a `blocked` state with a reason, never a silent retry.

What this buys: the conversation stays responsive while work runs; a task survives its conversation going quiet for a week; a crash loses no task because the execution is durable and its session is recomputed from the graph on the next turn; and "what is running right now" is a query over executions, answerable in the web UI and to the model.

3.3 The harness loop and the model loop

There are two loops, and keeping them distinct is what makes LOOP FOREVER cheap and safe. v0.8 sharpens the first one into an **event-driven execution model** (review: [notes/event-design-review.md](#)): the harness holds no in-flight state that a restart would lose, and it is quiescent between events by construction.

The harness loop is deterministic, never calls a model, and has no busy loop. It is parked on `select()` over its event sources: Discord, the completion sources (§3.16), the wake queue, tender health, budget thresholds, and a one-minute heartbeat timer. It has two very common states, and they differ only in how many open records the heartbeat has to reconcile:

1. **Quiescent.** No open executions, no in-flight work, no scheduled wakes. Zero cost between heartbeats.
2. **Tending.** No agent output to send and no human input to add, but work is in flight: a build in a shell, an ECS task, a wake due later, a confirmation outstanding. "Tending" is not a running loop; it is **the set of open completion records** in the WAL. The harness is just as parked as when quiescent. It stays fully responsive to humans, and **no model is called** until an event makes a model turn necessary: a completion arrives, a human speaks, a wake fires, a confirmation lands, a budget nears its ceiling, or the heartbeat reconciler finds something.

The point of tending is that ongoing work is exactly when responses must not be dropped. A task marked `running` is not a reason to rest; it is a reason for its completion to have a durable home that does not depend on anything in memory. The harness decides *when a model turn is warranted*; the model decides *what to do* in that turn.

The heartbeat is the level-triggered reconciler. Events are the fast path and can be lost (a receiver restart between dispatch and completion, a wrapper whose delivery failed and exited, a callback during a network flap, a cron that fired while the node was down). Every minute, without a model call, the heartbeat walks the open records: is there a spooled result, a queued message, a finished job scope, or an external status that says a record completed without us hearing? Is any record past its deadline? Is any wake due? Is any tender unhealthy, any budget window rolling, the Discord gateway alive? Findings become ordinary events; a completion the reconciler cannot establish becomes `outcome_unknown` and is surfaced to the principal. If nothing is found, it appends one `Heartbeat` record and returns to `select()`. This is the Step Functions task-token model and the Kubernetes resync lesson applied together: edge for latency, level for correctness.

Startup is reconciliation first, in this order: (1) restore the last checkpoint and replay the WAL tail, so every action record exists; (2) **stage** the completion inbox and spool without consuming anything; (3) for each staged completion with a matching record, commit settlement **and** the execution's

continuation state in one WAL transaction; (4) only then acknowledge or remove the completion entries; (5) reconcile remaining open records. Nothing in flight at crash time is lost or silently forgotten; it is either completed from durable evidence or reported as unknown.

The model loop is one execution taking one or more turns against the stateless Messages API. It runs only when the harness loop has an event for it. Its state machine:

```

IDLE -(inbound | wake)→ CLASSIFY → BUILD_CONTEXT → CALL_MODEL ─┬─ end_turn → JUDGE_STOP
                                                                └─ tool_use → GATE → E
JUDGE_STOP:    TERMINAL → reply, IDLE (human regains control)
                NOT_DONE → NUDGE (specific user-role message) → CALL_MODEL
JUDGE_CONTINUE: HEALTHY → CALL_MODEL
                THRASHING → INTERVENE (course-correction) → CALL_MODEL
                ESCALATE → ask in Discord, park until answered

```

- **CLASSIFY** is one Jev fan-out over the inbound turn: role called for, topics and people mentioned, security risk, whether it references another conversation, whether it is a follow-up fragment, continuation strategy hint.
- **BUILD_CONTEXT** assembles the prompt from the context graph (§4) under the budgeter, in cache-stable order (§4.4).
- **GATE** is the policy gate (§3.9) on each tool call.
- Each **JUDGE** is one Jev request over bounded state: the original ask, task-graph delta, last N tool calls and results truncated, turn count, spend, wall clock. Pack `loop.v1: work_state` (Choice: `complete` · `progressing` · `blocked_needs_human` · `thrashing` · `off_task` · `other`), `stopping_point_defined`, `same_action_repeating`, `cost_out_of_proportion`, `wants_human_input` (Nouls).
- **Execution outcomes** are distinct and deterministic where they can be: `complete` (accepted objective satisfied: $\text{Jev complete} \geq \tau$ and the execution's task scope has no open accepted tasks), `waiting` (no runnable work now; a wake condition exists, e.g. a due time, an external job id, a task blocked on another execution), `blocked` (progress needs a human), `cancelled` (authority or intent withdrawn via a deterministic control path, never via Jev), `failed` (recovery exhausted), `budget_exhausted` (reservation consumed; correct operation). LOOP FOREVER means: continue while authorized, runnable work exists within reserved resources.
- **Jev unavailable or malformed**: the loop treats the judgment as `abstain`; the execution finishes its current tool call, then parks as `waiting` on Jev recovery with a bounded retry, and tells the channel. It never continues autonomously without a judge and never invents an answer.
- **Wake** covers scheduled and self-scheduled continuation: the agent may ask to be woken ("check the build in twenty minutes"), which is a `Task` with a due time in the harness loop's wake queue, not a separate cron system. Running work is also a wake source: every shell job and external operation registers a completion event, so "the build finished" reaches the model loop as a turn, not as something a human has to notice and relay.

3.3a Loop, turn, and the Advancer

Three words the rest of the document leans on, fixed here.

Loop. One pass of the lifecycle: an input goes through the **toolchain manager** (which compiles or appends the context, §4.4a, and decides which tools are offered), a request is sent to the provider, and the model's response comes back, with text, tool calls, or both. A loop is the unit the ledger prices and the unit Jev sees. In a normal working turn there are many loops as the harness and the model chatter back and forth, executing tools and feeding results.

Turn. The sequence of loops run under **one acquisition of a session's turn lock** (§3.2a), from the stimulus that woke the execution to the moment the execution releases the lock. A turn ends when, and only when, the Advancer says so. This definition is chosen over "stimulus to reply to a human" because it lines up with everything else the harness already counts: the lock hold, the WAL transaction boundary (§4.6), the budget reservation, the "one turn per session" concurrency unit, and the append-or-recompile decision (one per turn). Under the event-driven model a long shell job splits what a human would call one exchange into two turns, one that dispatches and releases, and one that wakes on the completion; that is a feature, because each is a durable, resumable, separately priced record. When the human-perceived unit is needed (for the UI, or for the learning label "did this exchange succeed"), it is called an **exchange**: the run of turns in one session from a human stimulus to the next delivery addressed to a human. Anthropic's own "user turn / assistant turn" are called **provider messages** here, never turns, to keep the collision out of the code.

Advancer. The modular component that, after every loop, decides `continue` (execute the proposed tool calls, feed results into the next loop) or `end_turn(reason)`. It is a trait with pluggable policies:

- `stop_after_one_loop`: the first version's policy, and the permanent baseline. One loop, then the turn ends and the model's response is the turn's output.
- `until_no_tool_calls(max_loops)`: the conventional agent loop with a hard cap.
- `judged`: the spec's `JUDGE_STOP` (§3.5, §3.7), Jev deciding progress, stall, or done, with the deterministic controls (`/stop`, `/cancel`, budget, mechanical acceptance checks) always outranking it.

The Advancer never widens authority and never bypasses the gate: it decides whether to loop again, not what a loop may do. Its decision and reason are ledgered per loop, which is what makes the stopping policy learnable.

Turn trace. Every turn records a nested tree of timed spans as it runs: `turn > loop[n] > { compile, hook sites, provider.call > { first_byte, first_token }, advancer } > ... > session.write`, with the wait for the session's turn lock as the first child. Each span has a start and end in microseconds from the turn's start, a kind (turn, loop, hook, provider, mark, advancer, compile, store, lock), and attributes (handlers visited and outcome for a hook, request id and usage and stop reason for a provider call, decision for the advancer). The finished tree rides on the turn result, is written as a `turn.trace` ledger row, and on failure rides in the error payload up to the point of failure. This is the structure that tools, thinking, judgments, and completions will fill in as they arrive: a loop with three tool calls is three more spans under it, not a new mechanism. Rendering: a waterfall with a per-kind time summary in the web UI behind the timing link, and an indented tree with bars from `theseus ask --trace`. It is not sampling and it is not optional: the cost is a few microseconds per span, and the payoff is that every slow or strange turn can be read after the fact in exquisite detail.

3.4 Roles

One agent, many roles. The role table is a **living, versioned table** seeded before the ontology is fully known; rows are added as Jev or an operator discovers a new role, and the table's schema is `{id, stance, weights by node kind, hints, tools and shell classes favoured, verbosity, stop strictness, announce, added_by, added_at, version}`. The seed rows, drawn from six months of OpenClaw operation:

Role	Stance	Weights and hints
planner	goal-directed	tasks and decisions up; long horizon; asks clarifying questions before acting
coder	goal-directed	code, tool results, repo resources up; L0/L1 shells; terse; tests as evidence
reviewer	critical	diffs, prior decisions, conventions up; never edits; produces findings with evidence
operator (infra)	cautious	resources, events, runbooks up; destructive-tag awareness; prefers read tools first
researcher	exploratory	external resources, topics, citations up; longer outputs; flags uncertainty
thought partner	exploratory	people, preferences, prior conversations up; asks back; no tools by default
expresser	creative	persona and voice up; prose quality; minimal tools
triager	fast, decisive	events, tasks, people up; short outputs; routes rather than solves
secretary	goal-directed	calendar, people, events up; scheduling, briefings, follow-ups
security analyst	sceptical	provenance, trust labels, policy up; treats content as evidence not instruction
teacher	patient	deep knowledge and resources up; explains structure; checks understanding
archivist	curatorial	memory nodes, contradictions, supersessions up; proposes merges and forgets
(new)	(proposed by Jev or operator)	rows appended here; every addition carries <code>added_by</code> and a ledger reference to the classification cluster or human request that motivated it

Operators may define roles by hand in the web UI **and** Jev proposes new ones through the learning channel; both land in the same versioned role table. Role changes are **announced by default**: a short, in-channel note ("switching to reviewer") that is also a `Judgment`-linked event in the graph, so humans see the shift and can react to it, and the ledger can learn from those reactions. A binding may set `announce: false` to make role changes silent.

Beyond this set, the ontology grows to include Jev classifies the role each turn and scores the candidates. A role is a bundle of **values**, never filters: per-node-kind budget weights, personality guidance, stance (goal-directed, exploratory, learning), preferred tools and shell classes, verbosity, stop-criteria strictness. The role ontology is Jev-owned and grows through the learning loop: poorly covered classification clusters propose new roles; scored classifications flow to the operator on the learning channel (§3.10) and the web UI.

Jev's ontology, minimum: roles, people, topics, events, resources, memories, capabilities, deep knowledge. Expanding and honing it is a permanent activity.

3.5 Tasks (fluid)

The task graph is a core persisted structure whose interface is conversational. The agent sees the task graph **for its execution's scope** every turn (ids, titles, states, dependencies, owners, one-line acceptance), bounded by the budgeter: beyond the budget, the view collapses to open tasks plus one-line summaries of closed subtrees, with a count of what was elided and edits it through a reserved action namespace the harness executes, in the same tool-call grammar as everything else:

```
task.create {title, parent?, deps?, acceptance?, owner?, due?}
task.update {id, patch}      task.move {id, new_parent}      task.split {id, into[]}
task.merge {ids, into}       task.close {id, done|abandoned, evidence}
task.claim {id}              task.handoff {id, to: human|conversation}
```

Authoritative task state machine (the only one in this document): `proposed → accepted → in_progress → {blocked, waiting_human, suspended} → in_progress → done | failed | abandoned`. `suspended` is an execution-level pause (cancelled or budget-exhausted execution) that leaves the task recoverable. Human edits in conversation become the same actions. Discord shows the graph as a living pinned message per thread with components for common moves; slash commands cover work outside a conversation. Every mutation is a bus event and a ledger row. Three layers of the task graph have different mutability so the agent cannot redefine success:

- **Accepted objective and acceptance criteria:** authority-controlled. Changing them, or abandoning an accepted requirement, is a ledgered action that requires the requesting principal (or owner) to accept; the agent may propose it, never apply it alone. Abandoning is not completing.
- **Working plan and decomposition:** freely editable by the agent within the objective (create, split, merge, move, claim).
- **Evidence and outcomes:** append-only, tied to specific artifacts, test runs, or external states with their identity (commit, snapshot id, job id).

Where an acceptance criterion is mechanical (tests must pass, an artifact must exist, a check must be green), the deterministic check is a **necessary condition**: a failing check vetoes `done` while the requirement stands, regardless of what Jev says. Jev assesses the remaining semantic adequacy. This is not a second verification tier; it is the deterministic control path applied to acceptance.

`JUDGE_STOP` reads the graph: `done` is no open accepted tasks, every mechanical criterion satisfied, **and** Jev agreeing. Terminal and stalled states are first-tier Jev calls like every other loop state, escalated to a human only when the call is in question.

Concurrency. Channel serialization is not resource serialization; two executions in different channels can touch the same task or checkout. Task mutations are **versioned with compare-and-swap** (a stale version is rejected and the execution re-reads). Tasks have a **claim lease** with expiry; workspaces

have an advisory lock per execution with a documented conflict behaviour; test evidence records the exact snapshot identity it ran against.

3.6 Providers

Profiles. A profile names a provider, a model, an output limit, and a system prompt. Exactly one profile is **live**; it is chosen in config at startup and can be switched over the protocol at runtime, and the switch persists in the store so a restart keeps it. A turn may name a profile explicitly without changing the live one. Sessions and tasks will carry their own profile override in a later milestone; the resolution order is fixed now: raw overrides, then the turn's named profile, then the session or task override (future), then the live profile.

Providers are a table, not code (M0.5): every endpoint that speaks the Anthropic Messages API is a named `[providers.<name>]` entry with its base URL, key reference, and optional timeouts. The first-party API is the implicit `anthropic` entry; Z.ai's GLM series is `zai` at `https://api.z.ai/api/anthropic`. A turn selects provider and model explicitly or takes the configured default; both are ledgered on every call. A second wire protocol (OpenAI-compatible, Bedrock) would be a new `Provider` implementation behind the same trait and a new `kind` value.

`Provider` trait: `complete(request) -> Stream<Event>` with text, thinking, tool-use blocks, and usage. First and default implementation: Anthropic Messages API with streaming, adaptive thinking and `effort`, prompt caching (§4.4), and the Opus 5.5 / Fable 5.1 contract (thinking always on, no forced tool use, thinking blocks tied to model). Model per binding, with an optional Jev-driven cheap-first cascade (the OpenClaw jev-router design carried over). On provider outage Theseus fails closed: the conversation is told plainly that the model is unavailable, in-flight work is parked with its state intact, and nothing falls back to another provider.

When the provider does not answer (built in M0.5, 2026-09-25). Every call is bounded by four timeouts: connect, first byte (request sent to response headers), stream idle (longest silence between events), and total. Each failure is classified before anyone reasons about it: `timeout` (with its phase), `network`, `rate_limited` (with the provider's retry-after and rate-limit headers), `overloaded`, `server`, `auth`, `invalid_request`, `stream` (an error event mid-stream), `truncated` (the stream ended without `message_stop`). The class carries two facts the harness needs: whether a later identical call could plausibly succeed (`transient`), and whether the provider may have billed tokens we never saw (`usage_unknown`: true for idle and total timeouts, stream errors, and truncation; false for connect and first-byte timeouts, where nothing was generated). The turn fails, `provider.error` and `turn.failed` rows are ledgered, the error reaches the client with the class in `error.data`, and **nothing retries on its own**: a held reservation and a human or a later policy decide. The provider's request id, rate-limit headers, and timing (first byte, first token, total) are ledgered on every successful call as `provider.call`.

3.7 Judgments (Jev) as a core service

Judge trait, one implementation (TypeSafe Jev), always wrapped in a `Recording` decorator that writes every call to the ledger: pack id and version, state hash and size, answers with probabilities and confidence, latency, the action taken, and a slot for the later outcome label. Rules baked into the trait: atomic questions, mandatory no-match option, three-band confidence gate (act, confirm, escalate), state built by a `StateBuilder` that truncates to the model's limit by construction. Jev is never the sole gate for anything security-relevant.

Jev is a substantial dependency with correlated-error and latency exposure: the ~350 ms figure is one call, and a turn may make several, some serially dependent. The critical path is measured per workload class (simple reply, coding tool iteration, long-job completion, cross-channel recall, voice turn, large candidate set) and the budgeter batches fan-outs so serial depth, not call count, bounds latency. Judge probabilities are treated as **uncalibrated until the ledger shows otherwise**; thresholds start conservative and are tuned only on labelled outcomes.

Question packs in the design so far: `classify.v1`, `loop.v1`, `continuation.v1`, `shell.v1`, `memory.v1` (kind, durability, about, trust), `security.v1`, `sampling.v1`, `role.v1`. All versioned, all tuned by \$3.10.

3.8 MCP, both roles

As client. Transports: stdio for servers Theseus launches (in an L1 sandbox or an AWS class) and streamable HTTP for remote servers; no legacy SSE. Primitives: `tools`, `resources`, `prompts`, `elicitation`, `sampling`.

- Tools join the typed tool surface tagged `mcp:<server>` and pass the policy gate. Resources are readable via `resource.read` and pinnable into a binding's context.
- **Prompts** surface as slash commands scoped to the bindings the server is attached to; returned messages enter the turn as user-role content with a provenance label; templates are cached and diffed, and a change triggers an operator notice before reuse.
- **Elicitation** renders as Discord components (modal, select, buttons) to the human who owns the conversation; schema-validated answers only, a timeout that fails the call cleanly, only from servers the binding marks `interactive`, always recorded. Elicitation never substitutes for the policy gate.
- **Sampling** becomes a Theseus provider call under the conversation's model, spend ceiling, and ledger, judged by `sampling.v1` for proportionality and steering and passed through the three-band gate. Only servers the binding marks `samples` may sample.
- Credentials from Secrets Manager or SSM; OAuth for remote servers completes in Discord via a link component.

As server. Streamable HTTP bound to localhost only; Theseus never accepts inbound connections directly. Exposure, when wanted, is an external proxy's job. Auth is a single static API key for now; per-client scoped tokens are deferred. Exposed tools: `task.*`, `memory.search`, `memory.get`, `memory.propose` (goes through Jev ingest, never raw write), `conversation.open/send/status`, and any downstream tool the token's policy allows, re-exported through the gate. Exposed resources: task

graphs, policy-gated transcripts, read-only memory namespaces. Exposed prompts: persona-authored templates. Theseus-initiated sampling and elicitation toward its clients are supported for the case where the client is a human's front end. Every inbound call is a conversation event and a ledger row; rate and spend limits per token.

Excluded: `roots` from external servers (Theseus decides filesystem views), and any capability that lets a server modify bindings, policy, or credentials.

3.9 Policy, authority, and safety

Authority context. Every execution carries an explicit authority context, fixed at creation and re-validated at each tool call: the **principal** (the requesting human, the owner for scheduled or proactive work, or an MCP client identity for inbound MCP calls), any **delegation** (a human may delegate a bounded capability set for a bounded time), the **binding ceiling**, and the **resource ceilings** of the channel and guild. Effective permission is the intersection: no broader than what the principal holds, capped by the binding and resource ceilings, with explicit deny taking precedence over any allow. Authority is never derived from whoever happens to be present in a channel; an administrator and an ordinary user sharing a channel do not pool their powers.

Revocation. If a principal loses a Discord role mid-execution, the execution is re-validated on its next tool call and, if it no longer holds the needed permission, transitions to `blocked` with a clear message.

Derived work keeps its authority. A due-time wake, a task handoff, an execution restart, or a scheduled continuation **retains the initiating authority context and its delegation limits**. It never broadens. A user who cannot perform a privileged action now cannot obtain it by asking Theseus to do it tomorrow; the scheduled execution runs as that user and is `blocked` at the gate exactly as the live one would be. Owner-originated proactive work (binding-configured automations, heartbeat findings, consolidation) runs under a **separately declared owner grant** recorded on the binding, and an authorized person may explicitly **adopt** or reauthorize derived work to change its authority, which is itself a ledgered action.

Coalescing does not merge authority. When messages from several people are coalesced into one context build, authorship is preserved and, additionally, a message from a principal other than the execution's own is treated as a **new ask**: it either spawns its own execution under that principal's authority or, if it amends the current execution's accepted objective, requires an authorized scope amendment. A lower-privilege participant cannot steer an administrator's execution by typing into the same channel.

Channel switching intersects ceilings. An execution that moves to another channel keeps its original grant **and** must satisfy the destination binding's policy, disclosure rules, and resource ceilings; the effective permission is the intersection of both. It never carries a privileged origin binding into a less privileged destination.

Confirmation is not authorization. A confirm click proves intent for one exact action; it grants no capability the confirmer does not already hold. Confirmations are bound to the exact tool, arguments, target resource, policy context, and an expiry; a changed argument invalidates the confirm.

Gate. IAM-shaped policy inside Theseus. Tools carry tags: `read`, `write`, `destructive`, `spend`, `privileged`, `mcp:<server>`. Every tool call passes a three-band gate: allow, confirm, deny. Confirm goes to the requesting principal as a component on the message that would perform the action; for owner-authority executions it goes to the owner. On timeout nothing happens and there is no other fallback. Jev's `security.v1` feeds the risk score and never decides alone; adversarial content can move it.

Information flow. Cross-channel and cross-namespace recall is two decisions, not one: a **read** decision (may this execution's principal see nodes from that namespace or channel?) and a **disclosure** decision (may the result be shown in this channel to these participants?). Identity continuity for a `Person` namespace is not permission to disclose that person's data in a different guild or to other people. Both decisions are policy, evaluated deterministically, with Jev able to tighten but not loosen.

Enforcement is at compile time, not at output time. Once private material is in the model's context there is no reliable deterministic test of whether generated prose reveals it. Theseus therefore enforces disclosure **before generation** with **audience-safe context compilation**: the compiler (§4.4) receives the destination audience (channel, participants, external target) and admits only nodes whose confidentiality labels permit disclosure to that audience. Nodes carry a **confidentiality label** derived from their namespace and origin; generated nodes (`Message`, `Summary`, `Synthesis`, tool arguments sent outward, MCP responses and sampling requests) **inherit the most restrictive label of their inputs**, so an agent-authored summary of private or untrusted material is not public or trusted merely because its `origin` is `agent`. Declassification is an explicit, ledgered action by an authorized principal. The same labels drive the learning channel, web UI views, and logs, and they are what redaction lineage (§5.6) walks.

Provenance vs trust. `origin` on a node (operator, agent, tool, external, MCP server) is immutable provenance. `trust` is an inferred, mutable projection Jev may adjust. Jev never relabels origin.

Secrets. From AWS Secrets Manager or SSM Parameter Store when AWS is configured; from a local encrypted credential file with a passphrase or OS keyring in desktop mode. Never in `argv`; injected as environment at spawn; stdout scrubbed for secret shapes before it reaches the model or Discord. All actions land on the bus and in the ledger.

Default-safe is the operator's job, and the spec is honest about where the boundary really is: once an execution holds L0 with the operator's SSH agent and instance role, the effective security boundary is everything reachable with those credentials, not the harness's per-action tags. L0 is therefore `privileged`, opt-in per binding, and its use is loudly visible in the web UI.

3.10 Learning ledger and the learning channel

Feedback-driven optimization of our questions and parameters, not of the models, and not reinforcement learning in the technical sense: replaying a candidate against the incumbent's recorded outcome shows decision agreement, not what would have happened had the candidate acted. Every judgment that drove an action gets an outcome label from the human (reaction or slash command: wrong stop, should have stopped, wrong role, memory was useful or wrong), the system (a `complete` judgment followed by the same task reopening; a re-ask; a rehydration miss), or an offline audit by a

stronger model over a sample. A nightly job computes per-question precision and calibration, proposes wording and category changes as versioned packs, and evaluates them on **frozen, time-separated holdouts** with minimum sample sizes. Decision-quality evaluation (does the candidate agree with labels?) is kept separate from trajectory evaluation (did acting on it go well?), which only canaries can answer: a promoted pack runs on a small canary share with automatic rollback on safety regression. Changes touching `security.v1`, the privileged shell mapping, or authority decisions require human approval. The same machinery tunes `MemoryScience` parameters and the shell mapping table.

The **learning channel** is a Discord channel per guild (bound `listen_only` for memory, write-enabled for the ledger) where scored classifications, proposed new roles or categories, and pack promotions are posted for the operator to accept, reject, or nudge with a reaction. The web UI shows the same stream with richer controls.

3.11 Voice

Voice is a Discord voice channel via `songbird`, receive and transmit. STT and TTS are `Speech` plugins; Deepgram and Cartesia first, since ariadne carries the integration knowledge. Push-to-talk versus voice activity detection is a human Discord preference, not an agent concern. The agent joins a voice channel only when invited. Core behaviours lifted from ariadne: barge-in cancels TTS mid-sentence, proactive speech when background work reports back, deferred reports queued for the next pause, short verbal acknowledgements when a turn will take more than a couple of seconds. A voice channel is a channel; its conversation is shared with the paired text channel under the same binding; its transcript is text in the same graph. TTS voice is a persona attribute that roles may modulate.

3.12 Plugins

Two kinds. **Compiled-in feature crates** behind Cargo features: Discord, AWS, Anthropic, Jev, Deepgram, Cartesia, the native `MemoryScience`. One static binary; adding one is a rebuild and a pull request. **Runtime extensions** are MCP servers running in an L1 sandbox as children of the daemon's process tree (§3.8, §7): any language, isolated at the OS level, started in about a hundred milliseconds, hot-loaded and revocable without a restart. Contract surfaces: `Transport`, `Tool`, `Judge`, `Provider`, `MemoryScience`, `Speech`, `Shell`, `Store`, `Hook` (§3.17). Rejected: dynamic shared libraries (no isolation, break the static binary), deployment sidecars and Docker (tenders and sandboxed MCP servers are children of the same binary, not sidecars), embedded scripting engines (one language, weak isolation). **WASM was in earlier drafts and is not a commitment** (2026-09-26): it would buy microsecond starts and per-function capability grants at the cost of roughly doubling the dependency tree; it returns only as a measured experiment if per-tool process cost is shown to matter.

One tool contract, three backends (decided 2026-09-26, Appendix E). Every tool the model can call presents through one `Tool` contract: name, JSON schema, description, retry class (§3.16), the authority and capabilities it needs, and `invoke`. Two backends implement it and the model never learns which: compiled-in Rust, and MCP servers (out of process, in an L1 sandbox, §3.8). The gate, the ledger, hooks, the trace, and telemetry therefore see one shape and are written once. Vocabulary, so the layers do not blur: a **plugin** is a compiled-in unit that registers hooks, tools, providers, or channel adapters; **MCP** is the transport for tools that live out of process, and the only runtime

extension path. **Channels are adapters, never tools**: Discord is where authority comes from (who spoke, where, with which roles), and that context is trusted kernel data. Routed through a tool result it would become untrusted content under §3.9. An MCP server may expose Discord *actions* to the model (post, react); the inbound path and identity stay in the kernel. Likewise the kernel itself (executions, completions, the WAL, the turn lock), memory recall (compiler-selected, never model-invoked; an explicit lookup tool may exist alongside), and thinking (native to the model) are not tools.

3.13 Budgets

Budgets are a first-class notion: a `Budget` is a named ceiling with a unit (money, tokens, judgment calls, wall clock, tool invocations) and a window, attachable to any part of the system a binding, role, person, guild, conversation, task, MCP server, or shell class. The budgeter **reserves** against every budget in scope before a turn or tool call and settles actual consumption afterwards, atomically per execution, so concurrent executions cannot jointly overrun a shared ceiling. Admission control is global: new executions queue with bounded depth when node-wide reservations (model concurrency, Jev calls in flight, sandbox slots, arena headroom) are exhausted, and the channel is told it is queued. Overload sheds proactive and scheduled work first, human requests last. Budgets distinguish **enforceable limits** from **estimated exposure**. Reservations prevent concurrent executions from jointly admitting more work than permitted; they cannot by themselves guarantee a strict dollar ceiling when usage is unknown after a provider timeout, an external resource keeps billing while the harness is down, or cancellation is delayed or unsupported. Therefore: unknown consumption is treated conservatively (the reservation is held until reconciled, never released on timeout); Jev calls, hook handlers, memory and consolidation work, STT/TTS, retries, and compaction are accounted, not free; backend-enforced runtime limits (Lambda timeouts, ECS task limits, cgroup limits, wrapper deadlines) are set from the budget where available; a **reserved control and cleanup budget** exists so that reaching a ceiling never prevents cancelling work, recording outcomes, or telling a human; and elapsed lifetime, active compute time, and monetary spend are separate units. Disk-full handling likewise reserves capacity for control records and completion metadata so a running job can still write its result. Visibility exists from the first version: every turn result carries tokens in, out, cache read and cache write, first-token and total latency, and the provider request id; every session record accumulates its tokens; health reports totals and provider-error counts; the ledger is readable over the protocol (`ledger.tail`) and the CLI. Budget defaults will be ascertained as the system runs and recorded here as they are learned; until then, the only default is that a ceiling stops new work and says so. The owner sets budgets; operators may tighten them within their scope.

3.14 Web UI

Embedded in the binary, served on the node, authenticated by Discord OAuth against the operator role table. **First form (M0.5)**: a Vite + React app embedded in `theseusd` and served on `127.0.0.1:7433`, loopback only, no auth yet; the browser is a protocol client over a WebSocket where each text frame is one JSON-RPC line, so it has no privileged path into the kernel. It shows the prompt, the streamed reply, tokens in and out per exchange and per session, totals, timing, the classified error when a turn fails, and the notification stream behind each turn, where thinking and tool calls will render later. Purpose: immediate and local observability. Conversation snooping (live view of any conversation's

transcript, assembled context manifest, and loop state), the ledger stream with RL feedback controls, category and role management with scoring nudges, binding and policy editing with audit trail, tender health, arena occupancy, and budget burn. Historical search is CloudWatch: the ledger, bus events, and structured logs ship there through the durability tender when AWS is configured.

3.15 Executions

A **conversation** is a derived view (§3.2). An **execution** is the durable object that represents one piece of work being carried out. It is what the loop runs, what budgets are reserved against, what can be cancelled, and what survives a crash.

```
Execution {
  id, created_at, state: queued | running | waiting | blocked | cancelled | failed | budget
  session: { kind: conversation | task, root: channel id | task id },           // §3.2a
  channel: current channel it reports into (may change on switch), origin_channel, reports_
  authority: { principal, delegation?, binding, ceilings },                   // §3.9
  task_scope: [task ids accepted for this execution],
  role: current role, role_history,
  config_versions: { packs, model, memory params, shell mapping, binding revision },
  wake: { due_at? | external_op_id? | blocked_on_execution? | on_jev_recovery? },
  budget_reservations: [...],
  outstanding: [ confirmations, tool calls with idempotency keys, external ops ], // §3.1
  attempt: retry/recovery counters,
}
```

Rules: one execution per session, one turn at a time per execution; an execution reports into one channel at a time; a channel orders deliveries, not turns, so a conversation execution and any number of task executions reporting into the same channel run concurrently; the admission scheduler bounds how many hold a turn; a channel switch is an execution moving, not a new execution. An execution is `waiting` when it has no runnable model work but a wake condition exists (a due time, a running shell or external operation, a blocking execution, a pending confirmation, Jev recovery). `waiting` executions are tended by the harness loop at near-zero cost and never consume model or Jev calls until their wake fires. Deterministic control paths, `/stop`, `/cancel <execution>`, permission revocation, budget exhaustion, act on executions directly and never route through Jev. Executions are nodes in the graph (Execution kind) with `in_channel`, `by` (principal), `evidence_for` (tasks) edges, so the context assembler can show the model what it is currently executing and why.

3.16 External actions and completions

Local durability cannot make an external side effect atomic with the log, and holding a pending result in harness memory makes it die with the harness. Both problems have one answer: every tool call that leaves the process is a durable record whose completion arrives as an event from outside the harness's own stack.

Lifecycle, written to the WAL at each transition:

```
planned → authorized → dispatched → succeeded | failed | outcome_unknown
```

The `planned` record mints the **correlation id** and is committed before anything is dispatched; the `dispatched` record is committed before the call is made (transactional outbox). A crash between dispatch and result therefore leaves an open record for the reconciler, never a silent gap.

The `Completion envelope` is one type across all sources: `{correlation_id, outcome: succeeded|failed|unknown, result_ref (payload on SSD or S3, never inline beyond a cap), external_op_id?, started_at, finished_at, producer, signature}`. Handling is **idempotent**: a completion whose record is already settled is a logged no-op; a completion with no matching record is quarantined and surfaced, never inferred into a channel.

Transports, chosen per source, cheapest that preserves durability:

Source	Transport	Why
Native in-process tools that finish fast (reads, task and memory actions, most AWS reads)	synchronous return inside the tool loop	no ceremony for sub-second work; still a <code>planned / settled</code> pair in the WAL when the tool has side effects
On-node jobs (L0/L1 shells, PTY sessions, local tenders)	the job wrapper writes its result to the completion spool on the SSD, then signals over a Unix domain socket	survives harness restart; no port; no auth beyond filesystem permissions
AWS-side work (Lambda, ECS/Fargate, SSM, scheduled jobs, MCP servers running in AWS)	SQS long-poll fed by EventBridge and by the wrapper inside the job	pull, so "no inbound" holds; at-least-once with dedupe by correlation id
Sources that can do nothing but POST	loopback-only HTTP receiver, per-job HMAC, size-capped, off by default	the documented exception, never the default

The **job wrapper** is part of every shell class's contract (§7): it is **detached** (its lifetime does not depend on the harness; on the node it runs as its own systemd scope or L1 process tree), **durable** (result spooled to disk before any delivery attempt), and **cancellable** (the harness terminates it by correlation id through the execution's cancel path: kill the scope, stop the task, cancel the command). It is deliberately not "unkillable"; a runaway job must remain stoppable.

Deadlines and reconciliation. Every record carries a deadline from the tool's class and the execution's budget. The heartbeat reconciler (§3.3) checks open records against the spool, the queue, job-scope state, and, for AWS classes past their deadline, the service API. Reconciliation is event-first (EventBridge task state changes flow into the same queue) and polls only overdue records, so its cost scales with stuck work, not with total work.

Settlement is atomic with continuation. Accepting a completion writes, in one WAL transaction, the action's settled state **and** the owning execution's next durable state (runnable with the result queued, or waiting on something else). An in-memory mailbox notification is never the only link between a settled action and a waiting execution; if the process dies after the transaction, replay reconstructs the pending continuation. A duplicate completion is a no-op only because the transaction already happened, never because the first one was "handled" in memory.

`outcome_unknown` **is knowledge, not a terminal fact.** A record marked unknown stays **resolvable**: later authoritative evidence (a late completion, a reconciler finding) settles it to succeeded or failed and is ledgered as a resolution. Resolution never revives a cancelled execution and never authorizes new work by itself; it updates the record and, if the execution is still waiting on it, delivers the result.

Recovery never blindly retries a non-idempotent action. Correlation ids are not idempotency keys: an MCP or JSON-RPC request id correlates a request with its response and guarantees nothing about repeated effects; Discord nonces and AWS client tokens have their own validity windows. Every tool adapter therefore declares a **retry class** for each operation: `safe_to_repeat`, `idempotent_with_key` (naming the downstream key), `recoverable_by_external_id`, or `non_repeatable` (requires a human to resolve uncertainty). Generic SDK retry behaviour is disabled or overridden so it cannot silently contradict the class. Where the outcome cannot be established, `outcome_unknown` goes to the principal with the exact action, and the model gets it as a tool result so it can reason about it.

Cancellation is a lifecycle, not a flag. `cancel_requested` → `cancel_acknowledged` → `termination_verified`, or `cancel_unsupported` / `cancel_outcome_uncertain` where the backend offers no external termination (a running Lambda invocation, for example). Executions report which state they reached. Every job wrapper carries its **own deadline** enforced locally, so a harness outage never removes the only limit on a job's lifetime.

Provider and judge requests follow the same contract. An interrupted Messages API call may still be billed and its usage unknown; the record is settled as `outcome_unknown` for cost purposes and its reservation is held, not released, until reconciled. A partially streamed tool call is never dispatched: dispatch requires a complete, validated tool-use block.

Confirmations are bound to tool, arguments, resource, policy context, and expiry (§3.9); a confirm answered after a crash is re-validated against the record before the resumed call runs.

The simulator (§8) crashes at every transition, drops and duplicates completions, and restarts the harness mid-job as standing scenarios.

References, not payloads, between tools. Every tool result is a node with an id (§4.1). A tool that produces something large returns the node reference; the compiler decides how much of its text enters the model's context; another tool accepts the reference as input and reads the node directly. Raw bytes never round-trip through the prompt to get from one tool to the next, provenance and confidentiality labels ride with the node, and the ledger records the hand-off. Node ids are the pointers; no second URI scheme.

3.17 Hooks

Hooks are the extension and observation surface for everything the harness does. The design borrows the good parts of the Claude Agent SDK hook system (event taxonomy by cadence, structured results with per-event payloads, `deny` > `defer` > `ask` > `allow` precedence, `defer` for out-of-band human input, async observers, context caps, stop-override loop protection) and deliberately drops its footguns (exit-code control flow, fail-open timeouts on gates, best-effort filters as safety, shell scripts discovered from the working tree). Full design (types, dispatch, merge and failure rules, `defer` / `resume` mechanics,

event catalogue, examples, tests): `notes/theseus-hooks-design.md`. Investigation notes: `notes/claude-agent-sdk-hooks.md`; comparison with Strands, AgentCore Gateway interceptors, Bedrock Agents parsers, and OpenClaw: `notes/hooks-comparison.md`. Borrowed from Strands: one typed event object per hook with explicit mutable fields, reverse ordering for `After*` events, `projected_input_tokens` exposed before the model call, and `resume` as a first-class re-invocation mapped onto the harness-loop wake. Borrowed from OpenClaw: the per-kind failure-policy table, operator-tunable timeouts, and written merge contracts.

Invariants

- Hooks never widen authority. The policy gate (§3.9) decides what is permitted; a hook may tighten (deny, ask, defer, transform inputs) but an `allow` from a hook cannot override a policy deny or skip a required confirm.
- Hooks return typed results, never exit codes. A handler that fails to run, times out, or returns a malformed result is a ledgered error; for **gating** hooks that error **fails closed**, for **observer** hooks it fails open.
- Hooks are registered by compiled-in plugins, by sandboxed MCP servers, by protocol clients (observe only), and by the owner's binding config. Nothing in a workspace or repository can register a hook.
- Every hook invocation is a ledger row: event, handler id and version, input hash, result, latency. Hooks fire no hooks (recursion exclusion), and `JudgmentMade` is observe-only.
- Deterministic control paths (`/stop`, `/cancel`, revocation, budget exhaustion) are not hookable for veto; hooks may observe them.

Handler kinds: compiled-in plugin function; sandboxed MCP server in L1 (hot-loadable, the default for third parties and for the agent's own extensions); protocol client registered over the socket (observe only); HTTP endpoint on loopback only; **Jev pack** (a question pack whose answers map to a hook result, the Theseus version of the SDK's `prompt/agent` hooks); shell command, which runs as an ordinary `shell.run` through the shell classes and policy and is therefore a tool call, not a side channel. Observer hooks may be `async`; gating hooks may not.

Filtering: `matcher` (exact, list, or anchored regex) against the event's filter field, plus an optional typed predicate over the event payload. Filters select which handlers run; they are never relied on for safety.

Ordering. All outbound messages on a connection share one ordered queue, so a turn's notifications always precede its response on the wire. Hooks run **before final authorization**, never after it: `proposed call` → `hook transforms` → `schema validation` → `live policy and resource checks` → `confirmation bound to the final action` → `final revalidation` → `durable dispatch`. Any later change to arguments, target, tool, or policy context invalidates the confirm and reruns the checks. `ContextBuilt` additions go back through the budgeter, provenance labels, and compiler validation; `MessageSending` transforms cannot bypass disclosure checks; `PostToolCall.updated_output` never overwrites the immutable raw result or its success/failure status; a shell-backed hook skips recursive hook invocation but never policy or budget. Observer events are typed so that `continue: false` is not expressible from them; capabilities live in event-specific result types, not convention.

Result model: universal `continue: false + stopReason` on gating and transform events only (halts the execution; outranks everything); per-event `decision` payloads; `additionalContext` strings capped and spilled to a `Resource` node when large; precedence across handlers `deny > defer > ask > allow`; stop-override hooks carry an `override_active` flag and an eight-consecutive-continuation cap.

Events

Cadence	Event	Gating result
Harness loop	<code>Heartbeat</code> , <code>QuiescentEntered</code> , <code>WakeFired</code> , <code>TenderHealthChanged</code> , <code>BudgetThreshold</code>	observe
Execution	<code>ExecutionQueued</code> , <code>ExecutionStarted</code> , <code>ExecutionWaiting</code> , <code>ExecutionResumed</code> , <code>ExecutionCancelled</code> , <code>ExecutionEnded</code>	observe; <code>ExecutionStarted</code> may add context
Turn	<code>InboundReceived</code> (may add context or block with reason), <code>Classified</code> (observe: role, topics, risk), <code>RoleSwitching</code> (allow/deny), <code>ContextBuilt</code> (manifest visible; may add, veto sections, or block), <code>ModelResponse</code> (observe), <code>StopJudged</code> (observe; may override to continue with reason, capped), <code>ContinuationChosen</code> (observe)	as noted
Tool	<code>PreToolCall</code> (allow/deny/ask/defer, <code>updatedInput</code> , <code>additionalContext</code>), <code>PostToolCall</code> (<code>updatedOutput</code> , <code>additionalContext</code>), <code>PostToolCallFailure</code> , <code>PostToolBatch</code> (once per batch, may add context)	as noted
External action	<code>ActionPlanned</code> , <code>ActionAuthorized</code> , <code>ActionDispatched</code> , <code>ActionSettled</code> (succeeded/failed/unknown)	observe
Tasks	<code>TaskCreating</code> (block), <code>TaskUpdating</code> (block), <code>TaskCompleting</code> (block with reason), <code>TaskWoke</code>	as noted
Memory	<code>PreIngest</code> (may relabel kind/durability downward, or drop to floor retention), <code>PostRecall</code> (may remove candidates, never add), <code>ConsolidationProposed</code> (block)	as noted
Context	<code>PreCompaction</code> (block or choose strategy hint), <code>PostCompaction</code> , <code>PreRedaction</code> (observe), <code>PostRedaction</code>	as noted
Judgment	<code>PreJudgment</code> (may add state fields, never alter questions or answers), <code>JudgmentMade</code> (observe)	as noted
MCP	<code>ElicitationRequested</code> (answer programmatically, or route to Discord), <code>ElicitationAnswered</code> (override or block), <code>SamplingRequested</code> (allow/deny), <code>PromptExpanding</code> (block)	as noted
Discord	<code>MessageSending</code> (transform or cancel), <code>MessageSent</code> , <code>VoiceSpeaking</code> (transform or cancel), <code>ComponentInteraction</code>	as noted
Config	<code>BindingChanged</code> , <code>PolicyChanged</code> , <code>PackPromoted</code>	observe

`defer` on `PreToolCall` parks the execution as `waiting` with the pending call preserved; the confirm gate and MCP elicitation are both built on it, so a Discord component answer resumes the execution exactly where it stopped.

3.18 Wire protocol and client isolation

The core is a **server**. Nothing else in the system, not the CLI, not Discord, not the web UI, not the simulator, reaches into it except through one protocol. That is the isolation boundary Eddie asked for, and it is what keeps the kernel testable without a front end.

Protocol. JSON-RPC 2.0, newline-delimited JSON, one message per line, UTF-8. Requests, responses, and server-initiated notifications; requests are correlated by id, notifications carry the session id. Chosen over gRPC because it is what MCP, LSP, and ACP already speak, so every tool in the ecosystem can debug it with a terminal, and over a bespoke binary framing because message volume is token-bounded and the serialization cost is noise next to a provider call. All message types live in one dependency-free crate, `theseus-protocol` (serde types only), with a generated JSON Schema for non-Rust clients; if a binary encoding is ever needed, MessagePack over the same types is a framing change, not a protocol change.

Transports, same protocol on each.

1. **stdio**, when a client spawns the core as a child. This is the developer and test mode, and the way an editor or another agent harness drives Theseus (it is the shape of the Agent Client Protocol, and Theseus should be able to present as an ACP agent with a thin adapter).
2. **Unix domain socket**, the daemon mode and the real deployment: `theseus serve` listens on a socket in the state directory; many clients attach and detach while the core runs forever. Localhost only, by the settled reachability rule; file permissions are the authentication.
3. **In-process**, for adapters compiled into the binary (Discord, the web UI, tenders): the identical message types over a `tokio` channel. An in-binary adapter is still a client; it has no privileged path into the kernel.

Surface, first version. `session.open`, `session.list`, `turn.submit {session, input}`; notifications `turn.started`, `loop.started`, `model.delta` (streamed text), `tool.proposed`, `loop.ended`, `turn.ended {reason, output}`; `hooks.list`, `hooks.register`, `health`. It grows with the milestones (executions, tasks, ledger, confirmations), but the shape is set: requests change state, notifications report it, and every notification is also a ledger row.

Two binaries, one protocol (revised in M0 at Eddie's request: a server binary paired with a CLI binary). `theseusd` is the server: the daemon on a Unix socket, or `--stdio` when a client spawns it, plus `check` and `example-config`; `tenders` and `restore` join it later. `theseus` is the CLI: `ask`, `health`, `sessions`, `hooks list|watch`, `rpc`, `shutdown`, with `--json`, `--spawn`, stdin prompts, and shell exit codes (0 ok, 1 server or provider error, 2 usage, 3 cannot connect). The CLI links only `theseus-protocol`, never the core, so it cannot cheat. Both are static musl binaries.

3.19 Configuration and secrets

Opinionated, and simple. **Every secret lives in 1Password**, in the deployment's vault, and Theseus reads it at startup through a **service account**. The only secret the process may receive by any other path is the service-account token itself, from the environment or from a mode-0600 file whose path is configured.

- **Config** is a TOML document stored as a 1Password item (`theseus/config`) so the whole deployment is reconstructible from the vault. It may also be a local file for development; the schema is identical. Secret-valued fields are `op://vault/item/field` references, never values.
- **Resolution** happens once at startup and on an explicit `config.reload`; resolved values are held in memory in zeroizing containers, never written to disk, config, logs, the ledger, or a provider request except where they belong (an `Authorization` header). The redaction receipt system (§5.6) treats a leaked secret as must-not-exist content.
- **Mechanism.** The first version shells out to the `op` CLI (`op read op://...`) under the service-account token, because 1Password publishes no first-party Rust SDK; the community FFI wrappers around its C core exist and are the candidate for removing the `op` dependency later, once they are shown to build statically. References resolve concurrently at startup. The service account is read-only, so the config item is created by a human once; Theseus never writes to the vault.
- **Configuration is documented by its template, and the template is tested.** `theseusd example-config` prints a hand-written annotated TOML in which every parameter the code reads appears exactly once, set to its default or commented out with its default shown, with a line saying what it does. Three tests keep it honest: it parses and validates; a copy with every comment un-commented also parses under `deny_unknown_fields`, so no stale or not-yet-honored key can survive in it; and every key the loader can read appears in it, so no field can be added without documenting it. The consequence is a rule: config keys are not defined before code honors them; work not yet built is recorded in Part III, never as inert config. `theseusd config` prints the config actually loaded and its source, references only.
- **Token hygiene.** At startup Theseus checks the GitHub token against the API, logs its login, expiry, and days remaining, and warns when fewer than a configurable number of days remain (default 30). Never fatal.
- **Starting set** (vault `Eddie-Tabitha`, item names as they exist): `anthropic openclaw key`, `TypeSafe Jev key`, `zeroaltitude github PAT` (a fine-grained token with push on the owner's repositories, expiring 2027-02-18; chosen over the all-scopes classic token until Theseus is on rails), `z.ai key` (line `api key value`), and `strata-jam-aws-key`, a `label: value` note whose `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` lines are referenced separately (verified 2026-09-25 against STS: IAM user `stratajam`, account 560512680793). Discord and any others are added as their milestones arrive. The same posture applies to them all: GitHub and AWS credentials are read from 1Password too, never from `~/.aws` or `~/.config/gh`, and the process refuses to start if a referenced secret cannot be resolved (fail closed and say so).

3.20 Telemetry

OpenTelemetry is on by default and is a **projection of the record**, never a second instrumentation. The turn trace (§3.3a) is already a span tree with absolute start and end times; when a turn ends, Theseus walks the finished tree and emits it as OTel spans with those exact timestamps, so the hot path pays nothing beyond the trace it already records and the exported picture is byte-for-byte the ledger's. The mapping:

Theseus	OpenTelemetry
turn	root span; attributes <code>theseus.turn_id</code> , <code>theseus.session_id</code> , <code>theseus.profile</code> , <code>outcome</code> , <code>loops</code>
loop <i>n</i>	child span
provider.call	child span with the GenAI semantic conventions: <code>gen_ai.system</code> , <code>gen_ai.request.model</code> , <code>gen_ai.response.model</code> , <code>gen_ai.usage.input_tokens</code> , <code>gen_ai.usage.output_tokens</code> , <code>gen_ai.response.id</code> , <code>gen_ai.response.finish_reasons</code>
first_byte, first_token	events on the provider span
hook sites, compile, advancer, store, lock	events on their parent span by default; spans when <code>telemetry.hook_spans = true</code>
provider.error, turn.failed	span status error with the class, transient, and <code>usage_unknown</code> as attributes
turns, tokens, provider errors, durations	metrics: <code>theseus.turns</code> (profile, provider, model, outcome), <code>theseus.tokens</code> (direction), <code>theseus.provider.errors</code> (class, transient), histograms <code>theseus.turn.duration_ms</code> , <code>theseus.provider.call.duration_ms</code> , <code>theseus.provider.first_token_ms</code> ; <code>theseus.tool.calls</code> (family, tool, backend, outcome) and <code>theseus.tool.duration_ms</code> , from which the shell-fallback ratio (§3.23) is read

Transport is OTLP over HTTP/protobuf through the same `request` + `rustls` stack as the provider client; no gRPC, no C. Headers (a Honeycomb key, a Datadog key) come from the vault like every other secret. Resource attributes carry `service.name`, `service.version`, and `service.instance.id`.

Nothing leaves the process until `[telemetry].otlp_endpoint` is set; the pipeline is always compiled in and running so enabling it is a config change, not a build. Point it at a local Collector, Grafana Tempo, Honeycomb, Datadog, or the AWS Distro for OpenTelemetry, which is how "CloudWatch for historical search" (§1) is satisfied with no CloudWatch-specific code. The web UI and CLI keep reading the trace directly; OTel is for the fleet view.

3.21 Self-extension: planks, never the keel

Theseus may build its ship on the open ocean: add tools to itself while running. It may not touch the keel.

- **Planks** are tools. An `extend.propose` action lets the agent write a tool as an MCP server in any language, run it in an L1 sandbox, test it there, and produce a **manifest**: schema, capabilities requested, authority it needs, the tests it passed. Loading it is gated **deterministically** by an operator acknowledgement delivered as a Discord confirm (§3.9); Jev may advise, never approve. On ack the tool is hot-loaded with no restart, scoped to exactly the capabilities in its manifest, recorded as a versioned node with `derived_from` its proposal, ledgered, exported to

telemetry, and revocable with one command. `extend.propose`, the ack, the load, and the revocation are hook sites. A tool built by the agent is `trust: agent` and cannot request more authority than the execution that proposed it holds (§3.9 never widens).

- **Promotion to native** (`extend.promote`). A plank that has earned in-process speed or deep integration with the shell classes and the completion spool is ported to Rust inside the repository by the agent and opened as a **pull request**: implementation, tests, a Part III note. CI builds the static binary and runs everything; the operator reads the diff and merges; deployment is the routine graceful upgrade (§3.22), an operator action. Promotion can be one conversational request that yields a PR link, but the merge is human by default: native tools run in-process with kernel trust, and a bug or an injected malicious tool at that trust level owns the WAL, the secrets, and the policy store. A pull request is the only review artifact that can be read, tested, and reverted; an ack button for a compiled binary would approve something the operator cannot inspect. The repository *may* be set to auto-merge on green CI, which makes promotion fully agentic; that is a deliberate operator choice, off by default, to be made with evidence. Which planks earn promotion, and whether promoted tools are ever demoted, are ledger questions for the learning loop.
- **The keel** is the kernel binary: the gate, the WAL, the policy engine, the store, the trace, the compiler. The agent may propose changes to it only as pull requests to the repository, which pass CI and a human review and are deployed by the operator. It never self-applies, compiles, or restarts its own binary; "compile and restart yourself" is the control-plane tampering §6 warns about, and an ack dialog for it would ask the operator to approve code they have not read. A pull request gives them a diff.
- **When to extend** is a judgment, not a comprehension problem. The model can write a tool; deciding that a new tool is warranted rather than composing existing ones is a Jev pack (`EXTEND_WARRANTED`) plus the operator gate, and it is ledgered so the learning loop can see which extensions earned their keep.

3.22 Lifecycle: restart is routine

A restart of `theseud` is designed to be cheap, because nothing that matters lives only in process memory (§3.16, §4.4b). What a restart costs, exactly: any provider stream in flight at that instant, which becomes a classified `usage_unknown` failure with its trace, reservation held, turn resumable. Everything else is reconciled by the five-step startup (§3.3): finished jobs are picked up from the spool, running jobs are still running because their wrappers never depended on the harness, sessions and executions are records, the live profile is in the store, Discord resumes its gateway session.

Graceful upgrade is a first-class operation (`theseud upgrade`, or a signal): stop admitting new turns; let in-flight provider calls finish within a bounded window or fail them cleanly; flush telemetry; checkpoint the store; exec the new binary and hand it the listening sockets so no client sees a refused connection. Target: no lost work, no lost client connections, sub-second gap in turn admission.

Upgrade under load is a standing simulator scenario beside crash-at-every-boundary: a hundred sessions mid-turn, upgrade, every one settles correctly. Deploying a new keel is an operator action made cheap enough to do without ceremony; the agent never triggers it (§3.21).

This is a deliberate contrast with the gateway Theseus replaces, where in-flight tool calls, subagent handles, and session state live in process memory and a restart loses them. The M0 binary is not yet restart-safe in this sense; M1 Keel and M2 Kernel are where the construction happens, and the simulator is what proves it.

3.23 Toollets: native first

A **toollet** is a small, typed, in-process Rust tool behind the tool contract (§3.12): a few arguments with a JSON schema, structured output that becomes a node with provenance, microsecond to millisecond latency, no shell parsing, no PATH or environment dependence, a unit test. Theseus ships many of them and prefers them to shelling out everywhere.

Why this is a principle and not a taste. A `bash` string is opaque: the gate cannot tell `rm -rf` from `ls`, quoting is a permanent tax, the output is text that must be parsed back, and the ledger records only that "a shell ran." A native `fs.edit { path, find, replace }` has arguments policy can reason about, output that is already a node, and a trace span with real attributes. Every principle in §2 is stronger on a typed surface.

Families, each a crate:

Family	Replaces	Built on
<code>fs.*</code> read, write, edit, glob, grep, stat, tree	cat, sed, find, rg	the <code>ignore</code> and <code>grep</code> crates ripgrep is built from
<code>git.*</code> status, diff, log, blame, commit, branch, worktree	the git CLI	gitoxide
<code>text.*</code> diff, patch, json, yaml, toml, regex, hash, count	jq, diff, sha256sum, wc	serde, similar
<code>http.*</code> fetch, post	curl	reqwest
<code>gh.*</code> issues, pull requests, checks, reviews	the gh CLI	the GitHub REST API
<code>aws.*</code> per service	the aws CLI	the official Rust SDK
<code>task.*</code> , <code>memory.*</code> , <code>extend.*</code>	—	the kernel
<code>proc.run</code> typed argv, no shell	bash	tokio process; the escape hatch

Rules.

- Native does not mean unbounded. A toollet that does I/O or exceeds the synchronous bound (§3.16) is an action with a completion record like anything else; durability is unchanged.
- Native does not mean unscoped. A toollet runs with kernel trust, so it lands through the pull-request path (§3.21) and declares the authority it needs; the gate checks typed arguments, not strings.
- Tool count is the toolchain manager's problem, not the model's: toollets are offered by family and by the turn's needs (roles, Jev), and searched, so a hundred of them do not bloat every prompt.

- Every tool call is a trace span and a metric with family, tool, backend, and outcome. The **shell-fallback ratio** (`proc.run` over all calls) is watched; the most frequent `proc.run` argv patterns are the promotion queue for the next toollet (§3.21 `extend.promote`).
- A new capability arrives as a toollet unless there is a written reason it cannot; Part III records where this slipped.

3.24 The tool surface

Reviewed against Claude Code (about twenty tools, six of which do nearly all the work), Codex (seven native tools plus MCP; its `apply_patch` and `write_stdin` are the two ideas worth taking), and OpenClaw (seventy-odd top-level tools in a typical session, several with thirty-verb action enums, two memory systems, and operator controls exposed as model tools). Full review with verdicts in `docs/notes/tool-surface-review.md`.

Principles of the trim. One tool, one verb: no action enums. Typed in, node out: every result is a node with provenance, and composition is by node reference (§3.16). The kernel is not a tool: sessions, executions, cancellation, budgets, policy, config, secrets, and operator controls are protocol requests or deterministic commands. Memory is compiled, not called (§5), with one explicit lookup and one explicit note. The shell is reachable only through a typed argv and is counted (§3.23). Everything else is a plank (§3.12).

The selected set, thirty-five tools in twelve families, offered by family per turn so a coding turn sees perhaps fifteen schemas:

Family	Tools
fs	<code>read</code> (text, image, PDF as typed nodes), <code>write</code> , <code>edit</code> (exact-string replace with occurrence control), <code>patch</code> (unified diff), <code>glob</code> , <code>grep</code> , <code>list</code> (stat and tree)
proc	<code>run</code> (one-shot typed argv: cwd, env allowlist, timeout, shell class), <code>session.open</code> / <code>session.send</code> / <code>session.close</code> (a persistent interactive process: REPL, debugger)
text	<code>diff</code> , <code>query</code> (a jq subset over JSON, YAML, TOML nodes)
http	<code>fetch</code>
web	<code>search</code>
git	<code>diff</code> , <code>log</code> , <code>commit</code> ; the rest is <code>proc.run git ...</code> until the fallback ratio says otherwise
task	<code>create</code> , <code>update</code> , <code>move</code> , <code>split</code> , <code>merge</code> , <code>close</code> , <code>claim</code> , <code>handoff</code> (§3.5)
memory	<code>recall</code> , <code>note</code>
channel	<code>post</code> , <code>react</code> , <code>ask</code> (confirm, choose, or free text, to the requester or the owner)
node	<code>read</code> (any graph node by id, with a range: the reference-passing primitive)
wake	<code>at</code>
extend	<code>propose</code> , <code>promote</code> (§3.21)

Rejected, with the need's new home: free-form `bash` (→ `proc.run`); subagents, spawn, workflows (→ task sessions); todo and plan-mode tools (→ `task.*`); multi-action `message`, `browser`, `nodes` (→ `channel.*`; `browser` and device control are planks); fourteen memory tools (→ two); session, subagent, and automation tools (→ protocol requests, `wake.at`, `/cancel`); secrets, gateway, config, plugin tools (→ the operator's CLI and web UI, never the model); media generation, TTS, PDF, image viewing as tools (→ node types for input, planks for generation); skills as tools (→ roles and MCP prompts); notebook and worktree tools (→ `fs.edit`, `git.*`, snapshots).

The distinguishing claim is not fewer tools. It is that each tool is the whole of one idea, is typed enough for policy to read intent, and composes with every other through the graph.

4. The context graph

One master graph per deployment. Everything the agent could put in front of a model is a node; everything relating nodes is a typed edge. This is the session model, the transcript, and the memory system at once.

4.1 Nodes, edges, roots

- **Nodes:** `Message`, `ToolCall`, `ToolResult`, `Judgment`, `Task`, `Execution`, `Heartbeat`, `Person`, `Topic`, `Event`, `Resource`, `Capability`, `Role`, `Summary`, `Synthesis`, `Suppression`, `Redaction`, `Root`. Every node has an immutable **record** (`kind`, `origin`, payload reference, timestamps, authored edges) and a mutable **projection** (`retention` and FSRS state, `heat`, `trust`, task state, index membership) that is rebuildable from the record and the ledger. Append-only applies to the record; projections change freely.
- **Edges (typed, directed, timestamped):** `next`, `replies_to`, `in_channel`, `by`, `about`, `derived_from`, `summarizes`, `supersedes`, `contradicts`, `same_entity`, `mentions_conversation`, `in_role`, `evidence_for`, `judged_by`, `part_of`, `in_session`, `includes` (compilation → node, ranked), `reports_to`, `addressed_to`.
- **Multi-parent by design.** The graph is a DAG, not a tree. One `ToolResult` written by a task execution is `in_session` of that task, `in_channel` of the channel it was reported to, `includes`-d by any number of later compilations, and `summarizes`-d by a compaction. Each membership is its own edge; none is privileged. **Order is positional:** every record carries its monotonic WAL position, and any lineage (a channel's transcript, a session's tail, a compilation's contents) is a filter over membership edges sorted by position. `next` is a convenience edge for the per-channel fast path, never the definition of order.
- **Roots:** a node a lineage starts from. A channel's first node is its first root. A `Compilation` (§4.4a) is a root for the session that made it; a compaction's `summary` is a compilation whose `summarizes` edges point at the range it stands in for. The range is not removed; it is simply outside the default view. The graph never loses a node.
- **A context is a path selection.** The guaranteed default: follow `next` back to the nearest root within the channel. That is a plain transcript, served as a sequential read of an append-only per-channel log, never a graph traversal, and it must work with no Jev and no index.

4.2 Continuation strategies

1. **Transcript**: root → current. Always correct, always cheap.
2. **Ring**: transcript truncated from the front at the token boundary. Zero model cost.
3. **Compaction root**: summarize the range the ring would leave out into a `Summary` and re-root. One cheap call; the raw range stays in the graph and reachable.
4. **Assembled**: transcript tail plus a budgeted, Jev-selected neighbourhood: durable nodes, the task subgraph in play, borrowed summaries, participant `Person` nodes, the active role's hints.

Strategy 1 is what *appending* means; strategies 2 to 4 are what a *recompile* produces (§4.4a). Jev's `continue.v1` answers both questions at once: append, or recompile with which strategy, and whether a stronger model performs the compaction or synthesis. Strategies 1 to 3 need neither Jev nor the index.

4.3 Indexer and budgeter

The **indexer** turns "everything is eligible" into a few hundred candidates: exact indexes by channel, person, topic, and time; hybrid similarity through the index tender (§6). Incremental on every append. The **budgeter** allocates tokens across prompt sections and money across judgment calls per turn, weighted by the active role, aware of cache-read versus cache-write pricing, and logs every allocation for the ledger.

4.4 The context compiler

Graph selection yields node ids; the **compiler** turns them into a valid provider request. Its contract: preserve tool-call/tool-result pairing (never emit one without the other), preserve message ordering within a channel, honour model-specific constraints (thinking blocks tied to the model that produced them; no forced tool use on models that reject it), truncate only at message boundaries (the ring never cuts inside a message or a tool pair), and emit a **manifest** that records node ids, compiler version, renderer version, binding revision, model, and pack versions, so a turn can be reproduced faithfully from the ledger. The compiler takes the **session** (§3.2a) as its first input: the session's roots decide where traversal starts, so a task session compiles from its `Task` node outward (evidence, subtasks, the originating conversation as a secondary root, recalled memory) while a conversation session compiles from the channel's temporal view; both then pass through the same budgeter, labels, and manifest. The compiler is deterministic and unit-tested in the simulator against every strategy in §4.2. **Reproducibility** requires more than node ids because projections mutate: the manifest records an **as-of WAL position**, tool-schema versions, role and hook versions, pack versions, binding revision, model, and every prompt-affecting transformation, plus a **canonical request digest** so a reconstruction can be verified byte-for-byte. Judgment records likewise store the bounded state itself (or enough versioned references to rebuild it), not only its hash and size.

4.4a When a session is recompiled

Compiling every turn from scratch would be wrong twice over: it burns a prompt-cache miss on every turn, and it discards the plain fact that most turns in a live thread are simply the next thing said. So a session's context has two parts:

- a **compilation**: the prefix, produced by the compiler from the session's roots at some as-of position, persisted as a `Compilation` node (root kind) with its manifest. A compaction root (§4.2) is one kind of compilation; an assembled context is another.
- an **append tail**: everything the session has added since, in order, with no selection. This is the transcript strategy, and it is the default motion of every session.

Each turn the harness asks one question before the model runs: **append, or recompile?** The answer is layered so that the expensive judge is consulted only when something has actually changed.

1. **Deterministic triggers force a recompile** and never consult Jev: the audience or a confidentiality label in play changed (disclosure, §3.9); policy, tool schemas, role, or binding revision changed; the model changed; the tail would overflow the window or the configured tail budget; the session is new (a promoted task's first turn is always a compile); the execution glided to another channel; a redaction touched a node inside the current compilation.
2. **Candidate signals arm the judge**, cheaply and deterministically: a reference to another channel or an old topic (`mentions_conversation`, a recall hit outside the tail), a material task-state change in the session's scope, a long dormancy gap, a role hint change, a human asking for a fresh look, the tail crossing a soft length band, a cache-state change reported by the provider. If no signal fired, the turn **appends** and Jev is not called.
3. **Jev decides when a signal fired**: `continue.v1` receives the signals, the tail length, the cache state, the current compilation's manifest summary, and the budget, and answers `append` or `recompile(strategy)`. Jev owns this judgment as a core responsibility: it is deciding whether the world has changed enough that the model needs a rebuilt view rather than one more message.

Consequences. A long single-thread conversation appends turn after turn; its prefix is stable and cached; it behaves exactly like a normal transcript, because it is one. A task promoted from that conversation gets its own session and therefore its own first compilation, built from the task outward, then appends as it works. Diverse work is thereby compelled into distinct sessions, each compiled when it was needed and not again until something real changes. Recompiles are ledgered with their trigger, their cost (the cache miss, the compaction call), and the trajectory that followed, so the learning loop can tune the soft bands and Jev's threshold against outcomes: a recompile that did not change what the model did next was waste, and a stale append that preceded an error was a missed recompile.

Recompilation never loses anything. The old compilation, the tail, and the new compilation all remain in the graph with `derived_from` edges; the manifest of every turn names which compilation and which tail range it used, so any turn is reproducible.

4.4b How sessions persist across runtime restarts

A session is **durable by reference**, and that is the whole plan. Suppose a thousand sessions each hold a custom-selected context. Nothing about those thousand contexts is held in runtime memory that matters; each resolves in the store to a persisted **lineage**.

What is stored per session. One small record, updated in the same WAL transaction as the turn that changes it (§4.6):

```
Session {
  id, kind: conversation | task, roots: [channel id | task id, ...],
  execution_id,                               // §3.15, itself durable
  compilation_id,                             // current prefix
  tail: { from_position, last_position },      // append tail = in_session nodes in this
  strategy, budget_ref, labels_in_play, config_versions
}
```

Finding a session's own records. Positions are global across every session, so a `tail` position range is full of other sessions' records on a busy runtime. The index therefore carries a per-session ordered table, `session_id || position → ()`, and the tail walk is a range scan over exactly this session's records. The table is the same shape as the per-kind table and is rebuilt from the WAL like the rest of the index. (*Added v0.24, before M2 began; Session records are M2 work.*)

What a compilation is. A `Compilation` node whose record is the **selection**: ranked `includes` edges to the nodes it admitted, plus the manifest (§4.4: compiler and renderer versions, as-of position, model, binding revision, pack versions, request digest). Its rendered form, the provider-message bytes, is a **cache** stored alongside it and rebuildable byte-for-byte because the compiler is deterministic given the manifest. A thousand custom contexts are a thousand `Compilation` nodes that share the underlying nodes; no content is copied per session. At a few hundred references each, that is a few hundred thousand edges, which is nothing.

What a lineage is. `Session → Compilation → derived_from → Compilation → ... → first Compilation`, each carrying the tail range it was compiled from, so the full history of *how this session saw the world* is a chain of selections over one shared history. Promotion (§3.2a) is a branch in that DAG: a task's first compilation has `derived_from` the conversation's current compilation as well as `includes` edges into the conversation's nodes, which is why the graph must be multi-parent and why it is one.

Restart. The runtime reads Session and Execution records back from the store; nothing is "resumed" in memory. On a session's next turn the compiler loads the current compilation (the manifest, and the rendered cache if it is still resident on SSD), scans the tail by position, and proceeds with the append-or-recompile question exactly as if no restart had happened. The rendered prefix is reproducible, so a restart inside the provider's prompt-cache window still hits the provider cache. Sessions that are `waiting` cost a record each and nothing more; the resident graph (§6) rehydrates what a turn touches and only that.

What compile touches, and what it never touches. On the common turn the compiler loads the current `Compilation` by key (one index lookup), takes its rendered cache if resident or re-renders from its `includes` list (positioned reads, mostly arena hits), walks the session's tail through the per-session table (a handful of records since the last compile), and decides append or recompile (§4.4a). A recompile builds its candidate set from the retrieval indexes (§6.1) and adjacency scans bounded by the session's neighbourhood, then writes a new `Compilation` with `derived_from` the old one. Every step is a key lookup, a range scan, or a positioned read. The WAL is walked from a checkpoint in exactly two situations, crash recovery and index rebuild, and neither is on a turn.

Consequences for storage. Rendered compilation caches are the first thing tiering demotes, since they are rebuildable; selections and manifests are records and stay durable like everything else. Redaction (§5.6) of a node included by a compilation marks that compilation dirty, which is a deterministic recompile trigger (§4.4a), and restore applies tombstones before any rendered cache is trusted.

4.5 Prompt caching layout

Most stable first: persona and role hints; tool schemas; frozen transcript prefix from root to the last compaction; cache breakpoint; dynamic assembly; recent tail and new message. Compaction roots keep the prefix small and stable by construction. `continue.v1` receives cache state so it can prefer appending on hot conversations, and a recompile is scheduled at a natural boundary (after a tool loop closes, not mid-loop) whenever the trigger allows deferral.

4.6 What a turn writes

Inbound `Message` nodes; the turn trace (§3.3a); the `Compilation` node when the turn recompiled (selection plus manifest, and its rendered cache), otherwise a manifest reference to the compilation and tail range used; model output as `Message`; `ToolCall/ToolResult` with `in_session` and, when delivered, `in_channel` edges; loop `Judgment`s; `Task` mutations; and the updated `Session` record, all in one WAL transaction. The memory pass (§5) runs over these afterwards.

5. Memory

The graph is append-only. Compaction adds `summary` nodes and re-roots the default view; the trimmed range stays in the graph, reachable through `summarizes`, merely absent from that view. Decay lowers a node's retention and heat until the tiering tender moves its payload to cold storage, but the node, its edges, and its stub remain. Nothing is ever lost.

"Memory", then, is not a kind of node and not a survival test. It is **any node whose retention is high enough to be selected into a prompt**. Ingest is a durability decision over the nodes a turn wrote: how much retention each starts with, and which edges tie it into the graph. Not extraction of a separate object, and never a decision to discard.

5.1 MemoryScience trait, native implementation

```
trait MemoryScience {  
  fn gate(&self, candidate: &Node, neighbours: &[NodeRef]) -> GateDecision;    // store |  
  fn schedule(&self, node: &Node, event: AccessEvent) -> Retention;             // FSRS-6 u  
  fn activate(&self, seeds: &[NodeRef], budget: usize) -> Vec<(NodeRef, f32)>; // spreadi  
  fn decay_sweep(&self, now: Time) -> Vec<Demotion>;                          // candida  
}
```

Native v1: gate by embedding cosine plus exact about overlap for duplicates, Jev kind/durability for store-vs-drop, and a Jev Noul for contradiction between a candidate and its top neighbour; FSRS-6 stability and difficulty per node with dual-strength kept as two numbers; weighted BFS over typed edges with per-edge-type weights the role adjusts; decay by retention plus heat. Parameters are versioned data tuned by the ledger. A remote scorer is possible as a plugin; none is planned. Vestige's science informed this design; its code is not used (AGPL, SQLite-first).

5.2 The memory pass

1. Jev memory.v1 labels each new node: kind (preference · fact · decision · procedure · episode · transient · other), durability, about targets, trust.
2. Policy: transient and low-durability nodes stay heat-managed; secrets and external untrusted text never become durable without operator confirmation.
3. gate() assigns initial retention, merges by adding same_entity edges (the duplicate node remains; the edge routes recall to the canonical one), and flags contradictions with contradicts plus supersedes when the human is the source. "Drop" means starting retention at the floor so the node is cold from birth; it does not mean deletion.
4. Compaction Summary nodes take the same pass, so a summary can be durable while its range is not.
5. **Recursion exclusion:** Judgment and Heartbeat nodes, context manifests, and ledger records are never themselves candidates for the memory pass or for memory judgments. Only human and agent Message, ToolResult, Task, Summary, and Synthesis nodes are. Routine bookkeeping is deterministic, not judged.

5.3 Recall

BUILD_CONTEXT gathers candidates from the exact indexes and the hybrid similarity index, adds activate() neighbours of the seeds, dedups by id, fans out one Jev relevance Noul per candidate in a single request, then lets the budgeter allocate by node kind under the role's weights. Every recall records what was shown so outcome labels can flow back to both Jev tuning and schedule().

5.4 Consolidation (what remains of dreaming)

Re-scoring offline is unnecessary; selection is recomputed live. Synthesis is kept, as a memory-tender job: cluster nodes the ledger shows are recalled together, have a cheap model propose one synthesis per cluster, have Jev citation-check it against its sources, store accepted ones as Synthesis nodes

with `derived_from` edges, `trust: agent`, and the most restrictive confidentiality label of their sources. Shadow first, with a precise meaning: a shadow synthesis is *scored* (would the relevance judge have selected it; does an independent assessment rate it as supported and non-redundant) but its live *utility* cannot be shown while it is withheld. Promotion to live recall is a **bounded canary** measured on trajectory outcomes, exactly as question packs are (§3.10). No main-thread work, no unbounded replay.

5.5a Memory science must earn its place

FSRS models human recall; agent context selection has a different objective, selecting what improves the current task. The transfer is a hypothesis. There is also a known self-reinforcing loop: selection strengthens retention, strength increases selection, and repetition is mistaken for usefulness. Being shown is not being useful. Theseus therefore ships and measures a **baseline first**: transcript tail + task graph + summaries + BM25/embedding retrieval + deterministic freshness and provenance rules. Graph spreading activation, FSRS-style retention, Jev relevance reranking, learned role weights, and synthesized memories are each **ablated independently** against that baseline at a fixed total budget that includes judgment cost. Metrics: task success, false completion, unnecessary continuation, stale or contradictory recall, disclosure violations, latency, total cost. Retrieval agreement alone is not a success metric.

5.5 Namespaces, trust, forgetting

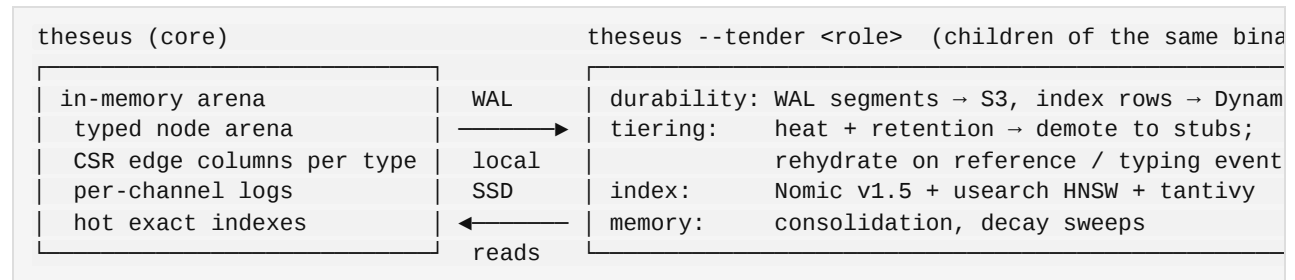
Namespaces: `person:<discord_user>`, `guild:<id>`, `channel:<id>`, `global`; bindings declare read and write sets; personal preferences always write to the person namespace. Trust labels on every durable node; external-derived nodes are recalled with their label and never gate policy. Forgetting is always an append: FSRS decay by disuse lowers retention; `supersedes` chains leave the superseded node in place so backward reach still works; operator `forget` appends a `Suppression` node that excludes its target from every view, every index, and tiering rehydration, with a receipt. Payload erasure is the single exception, specified in §5.6.

5.6 Redaction (the exception to append-only)

Suppression hides a node from views; it does not remove information that has already spread into summaries, syntheses, tool-result copies, ledger snapshots, indexes, embeddings, backups, and previously assembled contexts. For secrets and other must-not-exist payloads, Theseus permits **payload erasure with preserved structure**: the node record keeps its id, kind, origin, timestamps, and edges; the payload is replaced by an erasure marker (not a plain hash, which leaks low-entropy values), and a `Redaction` node is appended with a receipt naming who ordered it and why. Redaction is **lineage-aware**: it walks `summarizes`, `derived_from`, `part_of`, and `same_entity` edges to find descendants that may carry the content, flags each for re-summarization or erasure, removes the vectors and index entries, invalidates cached contexts that included the node, and records which backup segments contain the original so a backup policy (retention window, or targeted rewrite) can act. Low retention is not low persistence; a secret that slipped past ingest policy is redacted, never merely cooled.

6. Storage and tenders

Speed first, durability second, cost last.



- **The logical graph is permanent; the resident graph is a bounded cache over durable history.** Nothing about append-only requires anything to stay in RAM. Resident memory scales with the active working set, not lifetime traffic.
- **Arena.** Nodes by monotonic id; edge storage as immutable sorted segments per edge type with an in-memory delta, compacted in the background (compressed-sparse-row columns are a benchmark candidate for cold segments, not a commitment); per-channel logs as segments in a shared append file with an allocation index. Cold nodes leave RAM entirely, metadata and adjacency included, represented only by their id in a compact presence filter, and rehydrate from the SSD index on demand. Memory targets (§9) cover the whole process tree including tenders and loaded embedding weights.
- **Storage kernel (built in M1; Part III A1).** Two layers, one contract. The **WAL** is the truth: segment files of checksummed frames, one frame per append, one `fdatsync` per frame; a frame holding several records is atomic, which is how `settle(completion, continuation)` commits both or neither. Recovery verifies every frame, truncates a torn tail in the last segment, and refuses to guess at corruption anywhere else. The **index** is a rebuildable projection of the WAL in a pure-Rust embedded store behind an `Index` trait (`redb` chosen, `fjall` available): position → location, (kind, key) → latest position, (kind, position) for per-kind scans, and a checkpoint position. Index writes are non-durable; a checkpoint makes them durable; open replays the WAL past the checkpoint, so deleting the index entirely loses nothing. The arena remains the cache over this, never a second source of truth. Pure Rust keeps the static must build honest.
- **The turn lock and eventual durability.** "Speed first" is preserved by *where* the time goes, not by skipping durability. Within a channel exactly one turn advances at a time; that lock is held only while the core is doing local work. A turn is mostly waiting: a Messages API call is seconds, a shell job is seconds to hours, a judge call is hundreds of milliseconds, a human is minutes. At every such offload boundary the turn releases the lock and the core spends the surrendered time on **asynchronous durability work**: sealing the current WAL segment and handing it to the durability tender, taking checkpoints, flushing index updates, compacting edge segments, running the memory pass, uploading. The floor remains unchanged (intent is fsynced locally before dispatch); what changes is the off-node recovery point, which becomes **eventual: 5–60 s**

rather than 1–2 minutes, achieved for free from time the loop was not using anyway. The tender scheduler prioritizes by staleness: the oldest unshipped committed record bounds the current recovery-point exposure, and that number is exported as a metric and alarmed on.

- **WAL.** Every record appends to the local SSD and is fsynced on a short group-commit interval before the turn proceeds. Records carry a length prefix and checksum; a torn tail is truncated on recovery. Periodic **checkpoints** snapshot the arena so recovery is checkpoint plus tail, not full-history replay. Schema versions are stamped on every segment and migrations are forward-only transforms run by the durability tender. Disk-full is handled by refusing new turns with a clear message while tenders continue to drain. The SSD is a persistent volume that survives instance death and is encrypted at rest by the platform (EBS encryption, LUKS on a desktop), not by Theseus.
- **Tenders** consume the WAL and answer rehydration over a local socket; they never touch the arena directly. Durability ships to S3 and DynamoDB when configured, with a 5–60 s target measured as "age of the oldest unshipped committed record." Tiering demotes payloads by heat and retention (stub stays in the arena, payload on SSD and S3; nothing is removed from the graph), with a Jev backup opinion for lower heat bands; rehydration misses are logged. Index owns embeddings: 768-d stored, 256-d indexed, 768-d rerank; usearch memory-mapped from SSD; tantivy BM25; reciprocal-rank fusion then Jev relevance; asynchronous after commit; long nodes chunked with `part_of`. Memory runs consolidation and decay sweeps.
- **Completion pool.** A directory on the same SSD as the WAL where job wrappers write results before attempting delivery. Startup drains it before accepting events; the heartbeat reconciler reads it every minute. It is the reason a harness restart never loses a finished job.
- **Restore.** Rebuilding a node from S3 segments plus the DynamoDB index is a first-class, tested path from the first release (`theseus restore --from s3://...`), because S3 is presented as disk-failure recovery. Periodic automated drills remain deferred.
- **Embedding weights** are a versioned artifact fetched to the SSD on first run and pinned by hash, distributed separately from the static executable.
- **Desktop mode.** Same binary; tenders write to a local directory and SQLite; every AWS-side dependency is absent without error.

6.1 Graph state

Written 2026-09-26 in answer to Eddie's questions before M2. The durable layer and the structural index are conclusions implied by §4.4b and the kinds reserved in M1; the arena layout is a hypothesis to be benchmarked in M3 and is written here so the benchmark has something to confirm or overturn.

No graph database. The WAL is the truth for the graph as for everything else, and the graph is a set of projections over it. An embedded graph store (CozoDB, IndraDB) or SQLite would add a second durability story and a second recovery path beside the one proven in M1; a graph query language buys nothing when every traversal needed is "neighbours of X by edge type" or "lineage of session Y".

Durable layer. Nodes, edges, and compilations are ordinary WAL records (kinds `NODE`, `EDGE`, `COMPILATION`, `JUDGMENT`, reserved in M1). A node is keyed by its id, a UUIDv7 so ids sort by creation time. An edge is keyed `type || from || to`; its payload carries rank and weight. Both are append-only: a redaction or re-ranking is a new record carrying a tombstone or superseding flag, and

latest_by_key gives current state. A compilation record holds the manifest and the ordered includes list for fast rendering; the reverse includes edges, needed by redaction to ask "which compilations include this node", are written in the same frame. A turn that creates a node and a compilation with three hundred edges appends one frame; record count inside a frame costs microseconds and the frame pays the disk's one fsync.

Four projections, all rebuildable.

Projection	Store	Answers	Lag behind commit
Structural index	redb (\$6 storage kernel)	position → location; latest by key; per-kind, per-session, and adjacency range scans (type from to, plus a reverse column for includes and mentions)	none (same call, non-durable until checkpoint)
Resident arena	process memory	the working set: metadata columns, payloads, adjacency segments	none
Retrieval index	Index tender: usearch (256-d indexed, 768-d stored and reranked) + tantivy BM25, memory-mapped from SSD	"things about E" by paraphrase and by literal mention; reciprocal-rank fusion, then Jev relevance	seconds
Entity edges	Memory tender writing mentions and related edges back through the WAL	"things about E" as one adjacency scan once E is an entity node	seconds to minutes

Live traffic is arena first, structural index on a miss, and the WAL only as a byte source at a known offset. Nothing populates the whole graph into memory at startup; rehydration is lazy and per node, which is why a thousand waiting sessions cost a record each and no RAM. Writes go to WAL, index, and arena in one call, and only the WAL fsync is on the critical path.

Resident arena. A struct-of-arrays arena with a dense process-local handle per node and edges as sorted columns per edge type:

```

type Handle = u32;                                // dense, process-local, never persisted

struct Arena {
  ids:      Vec<NodeId>,                            // handle → uuid v7
  by_id:    HashMap<NodeId, Handle>,
  kind:     Vec<NodeKind>,
  position: Vec<u64>,                                // WAL position of the latest record
  loc:      Vec<RecordLocation>,                    // where the payload lives
  heat:     Vec<Heat>,                              // last-touch tick + count, for eviction
  payload:  Vec<Option<Arc<Bytes>>>,                // None = stub (evicted)
  free:     Vec<Handle>,
  edges:    [EdgeColumn; EdgeType::COUNT],
}

struct EdgeColumn {
  sealed: Vec<Arc<Segment>>,                        // immutable, sorted by (from, rank, to)
  delta:  Vec<(Handle, Handle, u32)>,                // unsorted appends since last compaction
  rev:    Vec<Arc<Segment>>,                        // sorted by (to, from); only for types that ne
}

```

Handles index every column directly, so a node's metadata is a few contiguous reads with no pointers or lifetimes; handles are never persisted and the uuid is the durable identity. Columns keep traversals cache-friendly and let a payload be evicted without disturbing the row. A neighbours query binary-searches each sealed segment and scans the small delta; compaction merges the delta into a new segment during turn-lock gaps, and readers hold `Arc<Segment>` so they never block the writer (one writer, the turn holding the lock; `arc-swap` on the segment lists, a short `RwLock` around the delta). An evicted node keeps its row and drops the payload; a node never touched in this process has no handle and is answered by the index until first touch. Not chosen: `petgraph` as the store (its value is algorithms; a throwaway view over a subgraph is fine for those), per-node `Vec<Edge>` lists (fragment, defeat eviction), and CSR in the first cut (the natural format for a sealed cold segment, and still a benchmark candidate). The arena sits behind a trait so the segment format can change under a benchmark without touching the compiler.

What does not scale, and where it is handled. WAL bytes grow forever by design; compaction of live-latest records into fresh segments and shipping sealed segments off-node are M5 Tenders, and until then `max_total_bytes` refuses appends rather than filling the disk. Index rebuild from scratch is linear in WAL size and is a recovery path, never a startup path. Edge payloads stay tiny (a large graph is millions of forty-byte edges, a WAL of a few hundred megabytes); node payloads carry the bulk and are what tiering demotes.

Durability boundary, stated plainly:

Failure	Guarantee
Harness process crash, SSD intact	Committed intent, results, and pending continuations recover exactly (WAL + spool)
Node restart, persistent disk intact	Same, plus job reconciliation; uncertainty reported where execution evidence is unavailable

Failure	Guarantee
SSD loss	Recover to the backed-up prefix with the stated 5–60 s recovery-point loss; actions in the gap may have happened externally without an intent record, so restore runs reconciliation of surviving external work with a conservative policy near the gap, and never reuses action identities after rollback
External side effect without recoverable evidence	Report uncertainty; never invent success, never blindly repeat

The DynamoDB index is rebuildable from S3 segments and is never a source of truth whose consistency recovery depends on. **Redaction receipts** distinguish completed local erasure, pending backup expiry or rewrite, and copies outside Theseus's control (already sent to Discord, a provider, or another service); restore applies redaction tombstones before restored content becomes visible.

Control-plane tampering: with L0 as default, code running under the operator's credentials could write the WAL, policy store, or spool. The runtime and its storage can run under a separate OS identity from L0 jobs (a dedicated `theseus` user owning the WAL, store, spool, and policy files, with L0 jobs running as the operator). This is an option, not a default, and the installer and documentation recommend it in strong terms: without it, L0 offers no protection of the record against the code it runs.

7. Shells

One tool family, many backends. Policy first determines the set of classes an execution's authority permits; `Jev shell.v1` then picks among that permitted set. Jev never widens the security boundary, only chooses within it. The model never names an instance.

```
shell.run {cmd, cwd?, env?, timeout_s?, workspace?, class_hint?}  shell.open/send/close (p
workspace.{create, attach_repo, snapshot, list}
```

Class	Where	Speed	Cost	Durable access	Isolation	Default for
L0 host	the node, its SSD	ms	included	full: source trees, SSH agent, caches, instance role	none; <code>privileged tag</code>	coding in known repos
L1 native sandbox	the node; Linux namespaces + cgroups + overlayfs, no daemon; contract below	~100 ms	small	read-only binds of chosen trees, scratch overlay	moderate, by contract	code the agent just wrote; package installs
A1 Fargate	ECS task per job, EFS or S3 workspace	20–60 s	per second, zero idle	EFS durable	strong	purpose-built one-offs; long batch

Class	Where	Speed	Cost	Durable access	Isolation	Default for
A2 Lambda	container image	~1 s warm	per invocation, 15-min cap	none durable	strong	short stateless jobs
A3 SSM	tagged fleet	seconds	fleet's	the host's	the host's	operating a specific host
A4 dev box	named EC2	seconds	its own	full, curated	the box's	curated environments

L1 contract (what "sandbox" means here, so it is not called strong by assertion): user, pid, mount, uts, ipc, and **net** namespaces; no network by default, with an explicit per-job egress allowlist and **no access to the instance metadata service or to localhost services**, including Theseus's own MCP server and web UI; capabilities dropped to none; a default seccomp profile; no device nodes beyond null/zero/random; masked `/proc` and `/sys`; cgroup limits on CPU, memory, pids, and disk with output size caps; the whole process tree killed on timeout or cancel. Anything the contract does not grant is denied. L0 grants everything the operator's user can do, and the spec says so plainly.

Default (decided 2026-09-25). Both L0 and L1 ship in the first useful agent. L0 is the early operator default; it is explicitly provisional and expected to be revisited once L1 has run real work for a while. Roles and Jev may steer a job to L1 within policy at any time; the default only decides what happens when nothing else has an opinion.

Every class launches through the **job wrapper** (§3.16): detached from the harness, result spooled before delivery, cancellable by correlation id. `shell.run` for a fast command still returns synchronously to the tool loop, but the record and the spool exist from the first millisecond, so a harness restart mid-command finds the result waiting rather than lost.

Workspaces point at existing directories on the node by default (the operator's real checkouts); cloning is an explicit attach for A1/A2. Snapshots everywhere. Credentials: L0 has the operator's agent and role; L1 gets nothing unless policy grants short-lived STS tokens or agent forwarding per job; AWS classes use scoped task roles. Persistent PTY sessions are `Resource` nodes owned by a conversation, reaped by an idle timeout the role may extend. A1–A4 light up only when AWS credentials exist.

8. Verification: the simulator and the replay harness

Two test targets are first-class from the first commit.

Replay harness. The ledger records every Jev call's state and answers, every context manifest, and every tool call. Replay re-runs recorded states through a candidate question pack, memory-science parameter set, or shell mapping and reports how decisions would have changed against the recorded outcome labels. It is how packs get promoted (§3.10) and how a bad judgment is reproduced from production without touching production.

Deterministic simulator. One process, no network, fully scripted, fast enough for CI:

- *Fake Discord*: a gateway that emits scripted events (messages, reactions, component clicks, voice joins) from a scenario file and records every outbound action Theseus takes, with a virtual clock so timeouts and wakes run in milliseconds.
- *Fake provider*: answers keyed by prompt hash from a recorded fixture set, or a small scripted policy ("call tool X then end_turn"), or, in a separate mode, a real cheap model for fuzzier scenarios.
- *Fake Jev*: recorded answers, or rule-based answers, or a real Jev call in a separate mode.
- *Real everything else*: the arena, WAL to a temp directory, tenders, indexer, budgeter, policy gate, MCP client against an in-process fake server.
- *Scenarios* are files: a binding, a cast of people, a script of inbound events, and assertions over the graph, the ledger, and the recorded outbound actions.

What it buys: invariants as property tests (a destructive tool never runs without a recorded confirm from the right person; the loop never exceeds a ceiling without a logged judge failure; a user `/model` pin is never overwritten; a compaction root is always reachable back to its range); regression tests for every incident, by turning a redacted production ledger slice into a scenario; and a way to develop the loop, roles, and memory pass for weeks before the Discord plugin is finished. It also gives the learning loop a dry-run: promote a pack in the simulator first, then in shadow, then canary, then live.

Standing scenario sets: crash at every external-action boundary (§3.16); lost completion (event never delivered, reconciler must find the spooled result); duplicate completion (second delivery is a logged no-op); completion arriving during harness restart; `/cancel` of a detached job and proof the scope died; completion with no matching record quarantined; a low-privilege user schedules a privileged action; crash after settlement but before continuation delivery; `outcome_unknown` followed by a genuine success; a late completion after cancellation; a hook mutation after confirmation; two executions on the same task or workspace; interrupted provider streaming with unknown usage; restore from a stale backup containing later-redacted content; a task promoted from a conversation runs while the conversation continues and a human message is routed to each correctly; a task waits a week for an answer and resumes with intact context; a hot thread of two hundred turns appends throughout with a stable cached prefix and a disclosure change forces exactly one recompile; a thousand sessions with distinct compilations survive a restart and each next turn reproduces its prefix byte-for-byte from the manifest; the admission ceiling is hit with a `/cancel` still honored immediately; authority edge cases (role revoked mid-execution, coalesced messages from two principals, confirm with changed arguments); redaction lineage; disk-full and Jev-outage behaviour. **First vertical slice:** one human request, one constrained typed tool action with a confirm, a task that goes `waiting` on a due time, a `/cancel`, and a crash during a dispatched action, all green in the simulator before any Discord code is written.

9. Efficiency targets (to measure, not assert)

Metric	Target
Binary size, static	under 60 MB with Wasmtime, AWS SDK, voice, search, and web UI; embedding weights are a separate artifact
RSS at 10,000 parked channels, 50 active executions	under 1 GB including arena metadata for the active set
Per-turn harness overhead (context compile + gate + WAL commit, warm arena; excludes model, Jev, tokenization, rehydration)	under 5 ms
Jev per turn	to be measured: calls, questions, input tokens, p50/p95/p99, per representative turn class; the ~350 ms / sub-cent figure is one call, and a turn makes several
Process start to accepting Discord events (checkpoint loaded, WAL tail replayed; excludes model/index warm-up, which proceeds in tenders)	under 2 s
WAL commit latency	under 5 ms at group-commit interval

10. Open questions

The three questions v0.9 held for Eddie are decided (§1: storage kernel, turn lock, shell default, control-plane separation). What remains open is deliberately the kind of question that only running code answers:

1. **Which embedded store.** `redb` or `fjall`, decided by the M0 benchmark in the plan, not by taste.
2. **The real recovery-point number.** 5–60 s is a target; the measured value under load, and whether the turn-lock model delivers it without stealing latency from turns, is an M3 result.
3. **Whether memory science transfers.** §5.5a's ablations are the answer; until then FSRs, spreading activation, and synthesis are experiments, not features.
4. **When L0 stops being the default.** Revisit after L1 has carried real work; the trigger is evidence, not a date.

Appendix A — Response to external review (GPT Astra, 2026-09-24)

Accepted and incorporated: an `Execution` record separate from the conversation (§3.15); concurrency defined per execution; an explicit authority context with intersection semantics and deny precedence, revocation handling, confirmation-is-not-authorization, read vs disclosure decisions, immutable origin vs inferred trust (§3.9); deterministic control paths and the six execution outcomes, budget exhaustion as correct operation, Jev-outage behaviour (§3.3, §3.15); the external-action lifecycle with outbox, idempotency, reconciliation, and bound confirmations (§3.16); record/projection

separation, cold metadata leaving RAM, checkpoints, torn records, schema versions, disk-full, a tested restore path, embedding weights as a separate artifact (§4.1, §6); payload redaction as the receipted exception with lineage-aware invalidation (§5.6); the memory-pass recursion exclusion (§5.2); the context compiler contract and full manifests (§4.4); the L1 contract and "Jev chooses within policy, never widens it" (§7); the narrower learning claim with holdouts, canaries, rollback, and human approval for security-adjacent changes (§3.10); atomic budget reservation, global admission control, bounded queues, overload shedding (§3.13); physical storage details demoted to benchmark candidates (§6); workload-defined performance targets (§9); the unified task state machine, `Suppression` and `Redaction` in the schema, task-graph scope limits, coalesced-message authorship, voice/text as linked separate channels, desktop-mode credentials, and the removal of the "no open questions" claim.

Pushed back or held for Eddie: L1-by-default (conflicts with a settled decision; §10.1); backup drills (compromise: restore path now, drills deferred; §10.2); role hints as enforcement (§10.3); per-client MCP credentials (Eddie chose a static key for now; §10.4).

Not adopted: nothing else in the review was rejected outright. The review's bottom line is taken as the sequencing constraint: the five contracts (execution, authority, external actions, storage and retention, resource accounting) and the first vertical slice in the simulator come before the custom storage and memory machinery.

Appendix C — Response to the all-webhook proposal (Eddie, 2026-09-24)

Adopted: the principle that nothing in flight lives only in harness memory; completion as an externally delivered event against a durable record; the harness quiescent between events; the heartbeat as reconciler; the detached job wrapper for every shell class; startup as reconciliation. Evidence: 24 lost subagent-completion wakes and hundreds of blocked-tool-call stalls in three days of our own gateway logs; Step Functions task tokens, Temporal activities, Kubernetes resync, Erlang mailboxes.

Changed in the folding: "webhook for everything" became "one `completion` envelope over the cheapest durable transport per source" (in-process, Unix socket spool, SQS pull, loopback HTTP as the exception), preserving the no-inbound posture and avoiding a receiver that must be up for anything to finish. "Unkillable wrapper" became "detached, durable, cancellable." Fast in-process tools stay synchronous inside the tool loop.

Appendix D — Response to the second external review (GPT Astra on v0.8, 2026-09-25)

Adopted and incorporated: the thesis reframed as a durable agent runtime with a provenance-aware context compiler (§0); derived work retains initiating authority, owner grants are separately declared, coalesced asks from other principals get their own authority, channel switches intersect ceilings (§3.9);

audience-safe context compilation with inherited confidentiality labels and explicit declassification (§3.9, §4.4); atomic settlement-plus-continuation, resolvable `outcome_unknown`, the five-step startup order, per-adapter retry classes, cancellation as a lifecycle with wrapper deadlines, provider and judge interruption accounting (§3.3, §3.16); hook ordering before final authorization and typed observer results (§3.17); the three-layer task graph with mechanical acceptance checks as necessary conditions and CAS/lease concurrency (§3.5); enforceable limits versus estimated exposure, held reservations on unknown usage, reserved control budget and disk capacity (§3.13); the budget-exhaustion label fix (§2); the memory baseline-and-ablation programme and the precise meaning of shadow (§5.4, §5.5a); the resident-graph-as-bounded-cache contract, reproducibility via as-of position and request digest, the durability boundary table, redaction receipt classes, and the tamper note (§4.4, §6); Jev critical-path measurement and uncalibrated-until-shown (§3.7); the eight additional simulator scenarios (§8); and retiring the "no open questions" claim (§10).

Held for Eddie, then decided 2026-09-25 (v0.10): embedded store as the day-one kernel with the arena as cache, plus the turn-lock model for eventual 5–60 s off-node durability; L0 and L1 both in the first agent with L0 the provisional default; control-plane separation as a strongly recommended option.

Taken as sequencing input, not spec text: the five-phase build order (prove the kernel in the simulator; ship a deliberately narrow agent; validate real failure boundaries; add intelligence features as measured experiments; expand the integration surface). It matches Appendix A's constraint and will seed the sequencing document.

Appendix E — Response to the "Omni-MCP" proposal (2026-09-26)

Eddie forwarded an essay arguing that a harness should treat everything as an MCP node: filesystem, memory, shell, sub-agents, thinking, channels, all speaking one RPC contract through a stateless router.

Adopted. The essay's fifth point is right and is the one worth keeping: one tool shape means authorization, approval, budgets, and tracing are written once. That became the tool contract with two backends (§3.12). Its fourth point, references instead of payloads between tools, is adopted natively with node ids as the pointers (§3.16).

Rejected, with reasons. Channels as MCP servers: Discord is the source of authority context, which must stay trusted kernel data (§3.9); MCP's request-response session model also does not fit a long-lived event source. The kernel as a stateless MCP router: MCP has no durable completion model, and a stateless router loses in-flight work on restart, which is the failure the execution kernel exists to prevent (§3.16). Memory as a tool: recall is compiler-selected every turn, never something the model must remember to ask for (§5). Thinking as a tool: native extended thinking exists; a tool adds latency and tokens. Sub-agents as nested MCP servers: the design has no multi-agent (§1); task sessions cover the need (§3.2a).

Where it led. Self-extension of tools at runtime under operator ack, with the kernel binary off limits to the agent (§3.21), and restart as a routine, tested operation (§3.22). v0.20 removes WASM as a commitment: the one runtime extension path is an MCP server in an L1 sandbox (§3.12, §3.21). v0.21

adds **NATIVE FIRST** (§2, §3.23): many small typed Rust toollets; the shell is the escape hatch and every shell call is a data point. v0.22 selects the tool surface itself (§3.24) after a review of Claude Code, Codex, and OpenClaw ([notes/tool-surface-review.md](#)). v0.23 closes **M1 Keel**: the WAL store exists, the crash test passes, and the engine is decided (Part III A1). v0.24 answers Eddie's questions on graph state (§4.4b, §6.1): nodes, edges, and compilations are WAL records; the graph is four rebuildable projections over them; compile never scans the WAL; the resident arena's layout is written down as an M3 hypothesis. v0.25 closes **M2 Kernel** (Part III A2): executions, actions, completions, the spool and job wrapper, budgets, admission, the five-step startup, group commit, and the deterministic kernel simulator. On whether the model would "get" it: the tool half, yes, deeply; the risks are tool-count bloat (dynamic tool search in the toolchain manager) and judgment about when to extend (a Jev pack plus the gate), not comprehension.

Appendix B — What is at stake in the default shell class

The question is only about the **default**: which class a coding-role shell command lands in when nothing else decides. L0 stays available either way, and policy can force either class for any binding.

What L0 as default buys. Speed and fidelity. Commands run at native disk speed on the operator's real checkouts with the operator's real toolchain, caches, SSH agent, and instance role. Nothing has to be bind-mounted, no overlay diverges from the real tree, git and package managers behave exactly as they do for a human at that machine, and interactive PTY sessions are trivially persistent. This is the OpenClaw experience today, and it is why the coding role is fast.

What L0 as default costs. The harness's per-action safety tags stop meaning what they say. `shell.run "npm test"` is tagged as a read-ish action, but what actually runs is whatever the repository's scripts, hooks, and dependencies do, with the operator's SSH keys and AWS role in the environment. A malicious or merely careless dependency, a prompt-injected file the agent was asked to read, or a plain model mistake can push to any repo the agent can push to, call any AWS API the instance role allows, or exfiltrate anything on the disk. The three-band gate still exists but it is gating the *command*, not the *effects*, and for arbitrary code those are not the same thing. Under L0-default the true security boundary is "everything reachable with the operator's credentials", and the reviewer's point is that the spec should not imply otherwise.

What L1 as default buys. The tags mean what they say again, within a written contract: no network unless allowlisted per job, no metadata service, no localhost (so no reaching Theseus's own MCP server or web UI), no SSH agent, no instance role, scratch writes in an overlay that can be discarded or promoted, the process tree killed on cancel. A repository hook that tries to phone home fails. A model mistake that runs `rm -rf` in the wrong place hits an overlay. The per-action gate becomes a real second layer rather than the only layer.

What L1 as default costs. Roughly 100 ms per command and real friction for the hot loop: source trees are read-only binds, so writes go to an overlay that must be promoted back to the real tree as an explicit step; credentials for git push or AWS calls must be granted per job as short-lived tokens, which

is more machinery and more prompts; some tooling misbehaves in namespaces (anything that wants the real user id, dockerd, some debuggers); and interactive sessions are less natural. Every one of these is solvable, and every one is a place where the agent will occasionally get stuck and ask.

The middle path the spec could take. Make the default depend on what the job needs, not on a global switch: L1 for anything Jev classifies as running code the agent wrote or installing packages (already the mapping); L0 for read-only exploration of known repos; and require an explicit, per-binding grant for L0 with write or credentials, defaulting the coding role's *writes* to L1-with-promotion. That keeps the fast path fast for reading and thinking, and puts the sandbox exactly where arbitrary code runs.

Decision (Eddie, 2026-09-24): L0 is the default; commit and see how it goes. Tabitha's recommendation, adopted: For BigHat, where the operator is the owner and the node is already trusted with these credentials, L0-default with the middle path's two exceptions (agent-authored code and package installs in L1, which is already the mapping) is the honest choice: it matches how we actually work and the spec now says plainly where the boundary is. For the open-source distribution, ship L1 as the default and make L0 the documented opt-in, because a stranger's default should be the safe one. The binding config already supports both; this is a defaults question, not an architecture question.

Part II — Build Plan

Tabitha, 2026-09-25. Part II says in what order the spec gets built, what each stage must prove before the next begins, and what is deliberately left out of the early stages. It follows the reviewer's five-phase shape (kernel → narrow agent → failure boundaries → intelligence as experiments → surface) and Appendix A's constraint: the execution kernel and simulator are the critical path, and everything else is a feature that must earn its place.

P0. How to read the plan

Each milestone below says what it will **build**, what must be **proved** before the next begins, and what is deliberately **not yet** done. What actually happened is recorded in Part III, one section per milestone, so a plan section is never edited to match reality after the fact; the divergence is written down instead.

There are no duration estimates. Milestones are ordered by what each must prove before the next can begin, and two things will dominate the pace: how much of the kernel the simulator forces us to rewrite (it always forces some), and how much time the Discord and Anthropic integration steals from the kernel if started too early. The plan defends against the second by refusing to start them until M2 is green.

Every milestone has three parts: **build** (what exists at the end), **prove** (the test that gates the next milestone, always executable, never a judgment call), and **not yet** (what a reasonable person would want to add here and must not). Milestones are Beads epics under `openc1aw-ph78`; each "prove" line becomes a closing criterion.

P1. Milestones at a glance

#	Name	One-line exit test
M0	First light	The binary starts with config and secrets from 1Password, all hook events registered with no handlers, and one <code>turn.submit</code> over the protocol returns the model's reply from exactly one loop
M1	Keel	<code>kill -9</code> at any point during a simulated workload; restart recovers every committed record byte-for-byte
M2	Kernel	All kernel scenarios in spec §8 pass under randomized fault injection, including crash inside each of the five startup steps
M3	First hands	Eddie completes a real coding task in a known repo from Discord; the harness is killed mid-job; the job finishes and its result lands in the channel
M4	Boundaries	Every row of the durability table (§6) is demonstrated, the measured off-node recovery point is under 60 s, and L1's contract tests pass
M5	Judgment	Jev-driven stopping and classification beat the deterministic baseline on a held-out trajectory set at equal total budget
M6	Memory as experiment	An ablation report over the §5.5a metrics says which of FSRS, spreading activation, reranking, and synthesis stay
M7	Surface	Theseus carries Eddie's daily Discord work end to end; OpenClaw is no longer in the loop for that channel

A usable agent exists at M3; the shape of the whole exists at M0. The back half is where the plan is least certain, and that is fine: by then the measurements exist to re-plan.

P2. M0 — First light

The very first version, specified by Eddie: a vertical slice through every layer, each at its thinnest, so the shape of the whole is real before any part is deep.

Build.

- Toolchain pinned: `rust-toolchain.toml` at stable (1.98.1 on 2026-09-25), target `x86_64-unknown-linux-musl`, static release profile, `cargo deny` with the permissive allowlist, `cargo nextest`, CI building the static binary on every push.
- Workspace crates: `theseus-protocol` (types only), `theseus-core` (kernel library), `theseus` (the binary: `serve`, `chat`, `--tender`).
- **Config and secrets (§3.19):** TOML config, `op://` references, resolution through the `op` CLI under the service-account token, zeroizing in-memory secrets, fail-closed startup. Starting set: Anthropic, Jev, GitHub, AWS (`strata-jam-aws-key`).
- **Hooks (§3.17):** every hook event defined as a typed enum with its kind (Gate, Transform, Claim, Observe), a registry that accepts handlers over the protocol (`hooks.register`) and in code, the run-hooks path wired at each event site, and zero handlers installed. The turn runs through

every hook site and nothing fires.

- **Turn runner (§3.3a):** session with a turn lock; a toolchain manager that compiles the context (the user prompt, nothing else), offers the tool list (empty), sends one provider request to the Anthropic Messages API with streaming, and returns the response.
- **Advancer:** the trait, with `stop_after_one_loop` as the only policy, ledgering its decision.
- **Protocol server (§3.18):** JSON-RPC over NDJSON on stdio and a Unix socket; `session.open`, `turn.submit`, `streamed model.delta`, `turn.ended`, `hooks.list`, `hooks.register`, `health`. `theseus chat` as the thin client.
- A first `store`: the session record and a per-turn ledger row in an embedded store, so even the hello slice persists what it did. No WAL discipline yet; that is M1.

Prove. From a clean shell with only the service-account token in the environment: `theseus serve` starts, resolves every referenced secret from the vault, and refuses to start if one is missing. `theseus chat` sends a prompt; the core runs one loop against the Anthropic API and returns the reply; the ledger shows one turn, one loop, `stop_after_one_loop`, and every hook site visited with zero handlers. The same conversation works over stdio and over the socket. The binary is static.

Not yet. No tools. No Discord. No Jev. No store durability guarantees. No context beyond the prompt.

P2b. M0.6 — Exquisite visibility (added 2026-09-25)

Not in the original plan. Eddie's principle, adopted as work before Keel because every later milestone is judged through it.

Build.

- The turn trace (§3.3a): nested spans on every turn, on the result, in the ledger, in failure payloads; waterfall in the web UI; `ask --trace`. (*Done, theseus-8af.*)
- OpenTelemetry as a default projection (§3.20): spans from the trace with exact timestamps, GenAI conventions on provider calls, metrics, OTLP/HTTP with vault-sourced headers, no-op until an endpoint is configured. (*theseus-vng.*)
- A standing rule for every later milestone: a new kind of work (tool call, judgment, completion, compaction, memory pass) lands with its span kind, its attributes, and its metric on the same commit; and a new capability lands as a native toollet unless there is a written reason it cannot (§3.23). Part III records where either slipped.

Prove. With a collector listening, one turn produces one trace whose spans match the ledger's `turn.trace` row exactly in count, names, nesting, and durations; the metrics for that turn arrive; with no endpoint configured, nothing is sent and the turn is no slower. Tested against an in-memory exporter; verified live against a receiver.

Not yet. Logs as OTel log records (the ledger is the log; it can be exported later). Prometheus scrape endpoint (optional, small).

P3. M1 — Keel

Build.

- CI extended to aarch64; the `--tender <role>` entry point that does nothing yet. (Toolchain, `cargo deny`, and the `x86_64` static build arrived in M0.)
- The event record types: `Node`, `Edge`, `Execution`, `Session`, `Compilation`, `Action`, `Completion`, `JudgmentRecord`, `LedgerRow`, with schema version stamps and forward-only migration hooks.
- The `Store` trait: `append`, `read-by-id`, `range-scan-by-position`, `checkpoint`, and a transactional `settle(completion, continuation)` primitive.
- Two `Store` implementations behind a feature flag: `redb` and `fjall`. A benchmark harness with our shape of workload: `append-heavy` small records with group commit, recent-window scans, id lookups, edge-segment reads, concurrent readers during writes.
- The WAL and spool layout on disk, with checksummed, length-prefixed records and torn-tail truncation.
- The simulator skeleton: virtual clock, deterministic scheduler, scripted fault injection (kill at record boundary, kill mid-fsync, disk full, torn write), and a replay checker that compares the recovered state against the oracle.

Prove. Under a randomized simulated workload, `kill -9` at any point followed by restart recovers every committed record exactly and no uncommitted record; the benchmark picks the store, and the number that picked it is recorded in the spec's §1.

Not yet. No Discord, no Anthropic, no tokio actors per channel, no arena optimization. The arena at this stage is a `HashMap`.

P4. M2 — Kernel

Build.

- Executions as durable objects with the state machine from §3.15 and an authority context (principal, grant, delegation limits, channel ceiling) that derived work inherits.
- Actions with harness-minted correlation ids, `ActionPlanned` → `Dispatched` → `Settled` | `OutcomeUnknown`, per-adapter **retry class**, and resolvable `OutcomeUnknown`.
- The `Completion` envelope and two transports: in-process channel and Unix-socket spool. SQS and loopback HTTP are stubs with the same interface.
- The job wrapper: detached systemd scope (or plain double-fork on the desktop), spooled result, own deadline, cancellable by id, with the cancellation lifecycle `requested` → `acknowledged` → `verified` | `unsupported`.
- The harness loop parked on `select()`, the one-minute heartbeat reconciler, the five-step startup order, at-least-once delivery with idempotent settlement.
- The turn lock per channel with explicit release points at offload boundaries, and a first durability job that runs in released time (checkpointing), so the model is exercised before it matters.

- Sessions as compiler scopes (§3.2a): one execution per session, per-execution turn locks, the admission scheduler, promotion by fork with inherited authority and carved budget, `reports_to` delivery as ordered channel actions.
- Budgets as **hard limits** with reservations, held reservations on unknown usage, and the reserved control-and-cleanup budget. No estimation yet.
- Deterministic policy gate with the ordering from §3.17 (transform → validate → policy → confirm bound to the final action → revalidate → dispatch), without hooks yet; the ordering is what is being tested.
- Ledger rows for every state transition.
- *Added 2026-09-26, before M2 began (v0.24)*: Session and Execution records in the store with the per-session position table (§4.4b); the `derived_from` edge written on promotion; the deterministic kernel simulator with virtual clock and fault injection, moved here from M1 (Part III A1); group commit, moved here from M1.

Prove. All kernel scenarios in §8 pass under randomized fault injection: lost completion, duplicate completion, completion during restart, cancel of a detached job, unknown then success, late completion after cancel, crash after settlement before continuation delivery, crash inside each of the five startup steps, two executions on the same task, wrapper deadline with harness down, a promoted task running concurrently with its conversation with messages routed to each, admission ceiling hit while `/cancel` is honored, graceful upgrade with a hundred sessions mid-turn. Reproducible from a seed.

Not yet. No model. The "tool" in M2 is a fake that sleeps and sometimes fails; the "channel" is a simulated mailbox.

P5. M3 — First hands

The narrow agent. One channel binding, one shell class, no intelligence beyond the model.

Build.

- Discord via `twilight`: one application, one guild, DM and one text channel, message send and edit, one component (the confirm button). Bindings as a file.
- Direct Anthropic Messages API with streaming, tool use, prompt-cache layout from §4.5, complete-block-only dispatch, and usage accounting into the ledger. Interrupted-call reservations held as unknown.
- A **model catalog** (`[catalog."<model id>"]`): per model, the serving provider, context window, maximum output tokens, prices per million tokens for input, output, cache read, and cache write, and capabilities (tools, vision, reasoning). A built-in catalog ships in the binary for the Anthropic family (`claude-opus-5-5`, `claude-opus-5`, `claude-sonnet-5`, `claude-fable-5-1`, `claude-haiku-4-5`) and the GLM 5.x models; config entries override or add. Three consumers: a profile that omits `max_output_tokens` defaults to the model's real ceiling; the budgeter uses the context window to decide what fits and when a recompile is forced; the ledger and telemetry turn tokens into dollars. An unknown model id still runs, with cost marked unknown and a startup warning.

Prices and limits cannot self-update (the provider's models endpoint lists ids, not limits or prices), so the catalog is a versioned table and every priced ledger row names the catalog version that priced it. Decided with Eddie 2026-09-26; it is config the moment code reads it, and not before (§3.19 rule). Until then `max_output_tokens` on a profile is the only token limit in config, and it is an output cap, never an input one.

- The context compiler in its simplest form: one compilation per session then transcript append; recompile only on the deterministic triggers of §4.4a (no Jev yet); a manifest that records the compilation, the tail range, the as-of position, and the request digest.
- **Toollets, native first (§3.23), from the selected set (§3.24):** the `fs` family (read, write, edit, patch, glob, grep, list), `proc.run`, `text.diff`, and `git.diff/git.log` on the operator's real checkouts, with `proc.run` as the typed, shell-free escape hatch through the L0 job wrapper; `proc.session.*`, `git.commit`, `text.query`, `http.fetch`, `web.search`, `channel.*`, `memory.*`, `node.read`, `wake.at`, and `extend.*` follow in their milestones. No `bash` tool: a shell is `proc.run { argv: ["bash", "-c", ...] }`, visible as such in the ledger. Fast in-process toollets stay synchronous; anything doing I/O past the bound or crossing the process boundary is an action with a completion. `theseus.tool.calls` and the shell-fallback ratio from the first turn.
- The model loop with deterministic control only: `/stop`, `/cancel`, budget exhaustion, confirm.
- The in-binary web UI in its first form: list executions, actions, and ledger rows; tail a channel. Read-only.
- `theseus.restore` from a local WAL directory (S3 comes in M4), because the restore path exists from the first release.

Prove. Eddie completes a real coding task in a known repository from Discord. During a long shell job the harness is killed and restarted; the job finishes, its completion is settled from the spool, the execution continues, and the result lands in the channel. The web UI shows the whole history. A request Eddie is not permitted to make is blocked at the gate with a clear message.

Not yet. No Jev, so promotion to an autonomous task is by explicit human command (`/task`) only. No roles. No memory beyond transcript. No MCP. No voice. No compaction (long conversations simply get a fresh transcript root by hand). This is the discipline Appendix A demanded and the first place we will be tempted to break it.

P6. M4 — Boundaries

Make the durability and safety claims true, and measure them.

Build.

- The durability tender: WAL segments to S3, index rows to DynamoDB, scheduled in released turn-lock time by staleness; the "oldest unshipped committed record" metric and alarm.
- `theseus.restore --from s3://...` with reconciliation near the gap and redaction tombstones applied before restored content becomes visible. Redaction with receipts (`erased_local`, `pending_backup`, `external_copies`).

- L1 native sandbox with the §7 contract, and **contract tests** that prove each denial: no route to the metadata service, no route to localhost services including Theseus's own UI, capabilities empty, seccomp active, process tree killed on cancel.
- Confidentiality labels on nodes with inheritance through generated nodes; audience-safe compilation; disclosure tests in the simulator (private material never reaches a public audience's context).
- Control-plane separation as an installer option: dedicated `theseus` user owning store, WAL, spool, and policy; L0 jobs as the operator.
- Cancellation verification per backend (systemd scope, L1 process tree), and `cancel_unsupported` reporting.

Prove. Every row of the durability table is demonstrated by a test: process crash, node restart with disk intact, SSD loss with restore from S3, external effect without evidence. The measured off-node recovery point under a synthetic load is under 60 s at p99 and the turn-latency cost of the durability work is reported. L1 contract tests pass. Disclosure tests pass.

Not yet. No AWS shell classes. No hooks. No Jev.

P7. M5 — Judgment

Jev enters, in shadow first, and hooks arrive because Jev packs are the first real hook handlers.

Build.

- Jev client with the confidence gate, state capping, batching by the budgeter, latency measurement per workload class, and every call accounted as spend.
- Question packs: `CLASSIFY` (new ask vs nudge vs control vs addressed-to-task, and when a conversation should promote to an autonomous task), `JUDGE_STOP`, `ROLE_GUESS`, `CONTINUE` (append or recompile, and how). All in **shadow**: recorded, compared with the deterministic baseline, never acting.
- The learning ledger with correct labels (`budget_exhausted` its own class), holdout split, and canary promotion of a pack from shadow to live when it beats the baseline.
- Hooks: `Gate`, `Transform`, `Claim`, `Observe` kinds with fail-closed gates, typed observer results, ordering before final authorization, ledger rows per invocation. Compiled-in handlers first; remote handlers over the protocol may observe.
- Roles table with the twelve seed rows, announced role changes, roles as hints in the compiler.
- Executions gain `waiting` on Jev recovery and provider outage (fail closed, say so).

Prove. On a held-out set of recorded trajectories, Jev-driven stopping and classification beat the deterministic-only baseline at equal total budget (judge cost included) on task success, false completion, and unnecessary continuation. Any pack that does not beat baseline stays in shadow and the plan says so.

Not yet. No memory science. No MCP.

P8. M6 — Memory as experiment

Build.

- The context graph beyond transcript: typed edges, roots, compaction roots (append-only, with `derived_from`), rotating ring, assembled continuation. `CONTINUE` goes live for recompile-strategy choice if M5 said it could.
- The index tender: Nomic v1.5 embeddings (768 stored, 256 indexed), usearch, tantivy, reciprocal-rank fusion. Baseline retrieval: transcript tail + task graph + summaries + BM25/embedding + freshness and provenance rules.
- `MemoryScience` trait with a **baseline** implementation (no retention model, no activation) and a native FSRS-6 + prediction-error + spreading-activation implementation behind it.
- The memory pass and recall as specified, consolidation as a tender job producing shadow syntheses with citation checks.
- Tiering tender: demote by heat, rehydrate on reference; arena as a bounded cache with the presence filter.
- The ablation harness: each feature toggled independently at fixed total budget, scored on §5.5a's metrics over recorded trajectories and a live canary.

Prove. An ablation report exists and is honest. Features that do not move task success, false completion, stale recall, or disclosure violations at equal cost are disabled by default and marked experimental in the spec.

Not yet. Voice, MCP, AWS shells, multi-channel gliding.

P9. M7 — Surface

Build.

- Multi-guild, multi-channel bindings; per-channel ceilings; gliding with intersected ceilings; coalescing with per-author authority; proactive and scheduled work under derived authority and owner grants; `wake` nodes.
- Tasks fluid in chat with the three mutability layers, CAS, claim leases, workspace locks.
- MCP client (tools, prompts, elicitation in), then MCP server on localhost with the static key, sampling budgeted and Jev-judged.
- Voice: `songbird` receive and send, STT and TTS as accounted spend, the voice turn as a workload class in the Jev latency budget.
- AWS shell classes A1–A4 with scoped task roles, SQS completion transport live, EventBridge task-state changes into the queue, reconciler API polling only past deadline.
- Web UI grows: in-thread observability, policy mapping administration, budgets, ledger views, learning channel.
- Self-extension (§3.21): `extend.propose`, the operator ack, hot-loading a sandboxed MCP server as a tool, revocation.

Prove. Theseus carries Eddie's daily Discord work end to end for two weeks with OpenClaw out of the loop for that channel, with the ledger showing budgets, judgments, and hook runs, and no disclosure or authority violation in the record.

P10. What is cut from the first useful agent, on purpose

Jev, roles, memory science, compaction, MCP, voice, AWS shells, hooks, multi-channel. M3 is a Discord front end on a durable execution kernel with a bash tool. If that is not already useful for coding in a known repo, the intelligence features will not rescue it; if it is, every later feature has a baseline to beat.

P11. Risks the plan is built around

Risk	Where it bites	Mitigation in the plan
Kernel rewrite after simulator findings	M2	Simulator exists before the kernel does (M1); the fake tool and mailbox keep the rewrite cheap
Integration work starves the kernel	M3	Discord and Anthropic are not started until M2 is green
Durability work steals turn latency	M4	Measured explicitly; the turn lock releases only at offload boundaries, and the tender is a separate process
Jev does not beat the baseline	M5	Shadow first; a pack that loses stays in shadow and the spec says so
Memory science does not transfer	M6	Baseline first, ablations gate every feature
L0 default proves unsafe in practice	M4 onward	L1 ships with contract tests in M4 so switching the default is a config change, not a project
Embedded store becomes the bottleneck	M6–M7	The <code>Store</code> trait and the M1 benchmark harness make the swap a bounded project

P12. Immediate next steps

1. Beads epics `theseus-9w9` (M0) through `theseus-ext` (M7) exist in the theseus repo, chained by dependency, each carrying its "prove" line.
2. M0 first steps: workspace crates, `rust-toolchain.toml`, `cargo deny`, the 1Password config loader, the hook registry, the turn runner, the protocol server, `theseus chat`.
3. Repository: `~/projects/theseus`, `github.com/zeroaltitude/theseus` (decided). This document lives there as `docs/the-ship-of-theseus.md` alongside the design notes.

Part III — As Built

The record. Each milestone gets a section when it closes: what exists, how it is proven, and a divergence table against Parts I and II. Entries are dated. Nothing here is aspirational; if it is not running, it is not in this part.

A0. M0 First light (closed 2026-09-25, theseus-9w9) and M0.5 Visibility (theseus-rbj, theseus-l32)

What exists. Repository `github.com/zeroaltitude/theseus`, dual-licensed MIT OR Apache-2.0, Rust 1.98.1 stable, static `x86_64-unknown-linux-musl` release builds (ring's C compiled by musl-gcc), `cargo deny` with a permissive-only allowlist, CI on every push (fmt, clippy `-D warnings`, tests, deny, web build with a dist diff, static build, artifact upload).

Workspace crates:

Crate	Role
<code>theseus-protocol</code>	Wire types only: JSON-RPC 2.0 over newline-delimited JSON; every payload struct. No runtime, no core dependency.
<code>theseus-core</code>	The kernel library: config, secrets, hooks, sessions, turn runner, Advancer, provider, store, ledger, RPC server.
<code>theseusd</code>	The server binary: daemon on a Unix socket, <code>--stdio</code> when spawned, embedded web UI, <code>check</code> , <code>example-config</code> .
<code>theseus</code>	The CLI binary: links only the protocol crate.

Protocol surface. Requests `health`, `session.open`, `session.list`, `turn.submit {session_id?, input, provider?, model?, hooks.list, hooks.register, hooks.unregister, ledger.tail {n?, kind?, session_id?}, shutdown`. Notifications `turn.started`, `loop.started`, `model.delta`, `loop.ended`, `turn.ended`, `hook.event`. Errors carry a JSON-RPC code and, for provider failures, `error.data {class, transient, usage_unknown, turn_id, session_id, elapsed_ms}`. Every connection has one ordered outbound queue, so a turn's notifications always precede its response.

Transports. Unix socket at `~/.theseus/theseus.sock` (mode 0600, stale-socket detection, refuses to steal a live one); stdio; WebSocket at `127.0.0.1:7433/ws` bridged through an in-memory duplex so the browser is an ordinary client.

Configuration and secrets. TOML, read by default from the 1Password item `op://Eddie-Tabitha/theseus-config/notesPlain`, or from a file via `--config/THESEUS_CONFIG`. Only the service-account token enters the process outside 1Password (env or a mode-0600 file). Every `[secrets]` reference resolves concurrently at startup through the `op` CLI or the process refuses to start, naming

the failing references. References accept a `#label` suffix selecting one `label: value` line of a multi-line note. Values live in zeroizing memory; `Debug` never prints them. The GitHub token is checked at startup (login, expiry, days left; warn under 30).

Providers. `[providers.<name>]` entries speaking the Anthropic Messages API, the implicit `anthropic` from `[model]`, `zai` at `https://api.z.ai/api/anthropic` in the example config. Streaming client with typed content blocks and stream events, tool input assembled at block stop, rate-limit headers, request id, first-byte/first-token/total timing, four timeouts (connect 10 s, first byte 60 s, stream idle 60 s, total 600 s), classified `ProviderError` with `transient` and `usage_unknown`, no automatic retry.

Kernel. Sessions as records with one turn lock each (re-read under the lock, so concurrent turns never lose updates); a toolchain manager that compiles the prompt alone and offers no tools; the `Advancer` trait with `stop_after_one_loop` active and `until_no_tool_calls` implemented but unused; 24 hook events with kinds Gate, Transform, Claim, Observe, every site visited per turn and ledgered, remote Observe handlers over the protocol; redb store with `sessions` and `ledger` tables; ledger rows for server start/stop, session open, turn start/end/fail, loop start/end, provider call/error, hook site visits, hook registration.

Accounting. Tokens in, out, cache read, cache write, first-token and total latency, provider request id per turn; cumulative per session; totals, provider-error count, and ledger size in health; `theseus ledger`, `theseus sessions list`, `theseus health`.

Web UI. Vite 8 + React 19, source in `web/`, built dist committed and embedded with `rust-embed`, loopback bind enforced, no auth. Prompt box and submit, streamed replies, per-exchange footer (provider/model, loops, stop reason, tokens, timing, request id), session and global totals, classified error display, collapsible event log per turn.

Proof. `scripts/smoke.sh` against the real API on debug and static binaries: check, health, hooks, streamed ask, piped `--json`, web served, ledger, stdio spawn mode, shutdown. 26 unit tests: SSE assembly, tool-input assembly, error classification, timeout phases, hook registry, secret reference parsing and line selection, Advancer policies, and RPC over an in-memory duplex (ordering, streamed deltas, every hook site ledgered, error codes, remote observer, same-session serialization, provider failure classification, usage accumulation, ledger tail, per-turn provider selection, parse errors). Live: provider connect and first-byte timeouts against a refused and a hanging endpoint; Z.ai reached and classified `rate_limited` (account unfunded).

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
One binary with subcommands (§3.18 originally)	Two binaries: <code>theseusd</code> and <code>theseus</code>	Eddie asked for a server paired with a shell-friendly CLI	Part I §3.18 updated; kept
Config item created in 1Password by Theseus	Created by Eddie by hand	The service account is read-only and cannot create items	Part I §3.19 updated: Theseus never writes to the vault

Planned	Actual	Why	Disposition
1Password read through an SDK	Shell out to the <code>op</code> CLI	No first-party Rust SDK; community FFI wrappers unproven on musl	Open: revisit when a wrapper builds statically
<code>#label</code> selection did not exist	Added to <code>op://</code> references	Real vault items are multi-line notes	Part I §3.19 updated; kept
M0 had no web UI, no provider table, no timeouts, no usage rollups	All four built as M0.5 the same day	Eddie wants visibility and a failure story from the first version	Part I §3.6, §3.13, §3.14 updated; plan not rewritten
One store per node	stdio mode uses <code>theseus-stdio.redb</code>	redb is single-process; a spawned server must not fight the daemon	Kept for now; M1 Keel decides the store layout
L1 shell not in M0	Still not built	As planned	none
Continuing a session carries context	Prompt only	As planned for M0; transcript continuation is M2	none
Web UI authenticated by Discord OAuth	No auth, loopback only	First form; auth arrives with bindings in M7	Part I §3.14 says so
musl build via musl-tools from the start	Host gcc first, musl-gcc after Eddie provided sudo	No passwordless sudo on the node	Resolved

Added after M0.5 (theseus-rfl, 2026-09-25): profiles. `[profiles.<name>]` with provider, model, `max_tokens`, system; the implicit `default` profile is built from `[model]`; `[model].live` names the startup profile; `profile.use` switches at runtime and persists in the store's `meta` table (a persisted switch wins over config on restart unless it names a profile that no longer exists, in which case config wins and a warning is logged); `profile.list` reports the live name and whether it came from config or runtime; `turn.submit.profile` runs one turn under another profile; the CLI has `theseus profile list|use` and `ask -P`; the web UI has a live-profile selector in the header. Every turn result and ledger row names its profile. Test: switch, route, explicit override, persistence across a fresh core over the same store.

Added the same evening (theseus-8af): the turn trace. `Trace` builder in the core (enter/exit/mark/record/finish), `span` in the protocol, every hook site visit recorded as a span with its handler count and outcome, provider calls as spans with first-byte and first-token marks and request id, usage, and stop reason as attributes, the advancer decision, the lock wait, and the session write. On the result, in a `turn.trace` ledger row, and in `error.data.trace` for failed turns. Web UI: the timing link opens a waterfall with a per-kind summary and click-for-attributes; CLI: `ask --trace`. Test asserts the tree's shape. Observed on the first real turn: 1.35 s total, of which 1.25 s was the provider (first byte at 704 ms, first token at 846 ms) and 4.6 ms the session write; every hook site under 15 µs.

Added the same night (theseus-vng): OpenTelemetry as a default projection. `core::telemetry:` OTLP/HTTP (protobuf, reqwest + rustls) span and metric exporters built only when `[telemetry].otlp_endpoint` is set, otherwise a no-op that costs nothing; the finished turn trace is walked into OTEL spans with the recorded timestamps (root placed by `origin_unix_ms`, which the trace

now carries); provider calls are `client` spans with the GenAI attributes (`gen_ai.provider.name`, `gen_ai.request.model`, `gen_ai.response.id`, `gen_ai.response.finish_reasons`, `gen_ai.usage.*`); hook sites, marks, compile, store, lock, and advancer are events on their parent unless `hook_spans = true`; failed turns export with error status and the partial trace; metrics `theseus.turns`, `theseus.tokens`, `theseus.provider.errors`, `theseus.turn.duration_ms`, `theseus.provider.call.duration_ms`, `theseus.provider.first_token_ms`; headers from a vault secret; resource `service.name/version/instance.id`; flushed on shutdown; health reports the endpoint. The upstream semantic-conventions crate deprecated its GenAI constants (they moved to a separate repository), so the attribute names are pinned locally. Five tests against in-memory exporters (nesting, parent ids, exact timestamps, events vs spans, error status, metric names, header formats, disabled no-op). Dependency cost: ~185 crates in the core against ~140 before.

Reversal, 2026-09-26: WASM out. Earlier drafts named WASM components (`wasmtime`) as the hot-loadable plugin form and even scheduled a "WASM plugin ABI" for M7. Eddie asked why; the honest answer was that it bought microsecond starts and fine-grained capability grants we have not shown we need, at the cost of a very large dependency and toolchain churn, while the L1 sandbox plus MCP, both already specified, give the same isolation and hot loading for free. The single runtime extension path is now an MCP server in an L1 sandbox; WASM returns only if measured per-tool process cost demands it. Recorded here so the reasoning survives.

Finding, same day: the vault's `z.ai` key item is not the key OpenClaw uses. Fingerprints differ; the vault key returns 429 code 1113 (insufficient balance) and the OpenClaw key (`models.providers.zai.apiKey`, shared by every agent including Tank) returns 200 with a GLM reply. Eddie to update the vault item; Theseus reads only from the vault by design, so no GLM reply has been observed through Theseus yet.

Known gaps carried forward. Continuing a session sends the new prompt only. Remote hook handlers can observe but not gate or transform. Cost in dollars is not computed (tokens only; a pricing table per model is a later addition). The web UI has no model selector yet (the CLI has `-p/-m`). The `Z.ai` account has no balance, so no GLM reply has been observed end to end.

A1. M1 Keel (closed 2026-09-26, theseus-cfl)

What exists. Crate `theseus-store`: `NewRecord/Record` with kind and schema version (kinds: session, ledger, meta, execution, action, completion, node, edge, compilation, judgment, checkpoint); `wal` with 64 MB segments, frames `MAGIC | len | crc32 | body{count, records}`, one `write_all` and one `fdatsync` per frame, a `max_total_bytes` cap that fails appends cleanly (the disk-full case), recovery that verifies every frame, truncates a torn tail only in the last segment, and refuses (`Corrupt`) if an earlier segment is damaged; `Index` trait with `redb` and `fjall` implementations over four tables; `walstore` composing them with atomic `append/settle`, `get`, `scan`, `latest_by_key`, `latest_of_kind`, `tail_of_kind`, `count_of_kind`, `checkpoint`, `stats`, a manifest that pins the engine a directory was created with, and index rebuild from the WAL past the checkpoint on open. Crate `theseus-sim`: `worker` (appends random 1–4-record frames of 16 B–32 KB, printing each committed position and payload `crc` only after the append returned), `crash-test` (spawns the worker, SIGKILLS it after 15–250 ms, in 70 %

of runs tears the WAL tail by truncation, junk, or a flipped byte, reopens, verifies every reported record byte-for-byte, contiguity, and that appends continue; three restarts per iteration), and `bench`. The M0 core now writes through `theseus_store ([server].store_engine, default redb)`, checkpoints on shutdown, and reports engine, last position, and WAL size at startup.

Proof. 47 workspace tests, including the WAL round trip, torn tail truncated with earlier frames intact, corrupt middle frame refused, two-record frame atomic (both or neither), full WAL refusing cleanly, both indexes, both stores, and index loss rebuilt from the WAL. The exit test: `theseus-sim crash-test --iterations 40 --restarts 3` on each engine, 120 kill-and-recover cycles per engine with tail tearing, **zero committed records lost**, 255 KB and 141 KB of torn bytes removed, contiguity held every time. Smoke test green on the migrated daemon. Static musl build green.

Benchmark (this node, WSL2, one writer, 20 k–50 k records):

	redb	fjall
append, fsync per frame	142 frames/s, 7.1 ms/frame	145 frames/s, 6.9 ms/frame
append, no fsync	17.8 k frames/s	30.5 k frames/s
get by position	374–381 k/s	279–351 k/s
latest by key	428–452 k/s	475–521 k/s
newest-50 of a kind	8–11 k/s	8.7–10.3 k/s
latest of every key (50 sessions)	0.12 ms	0.89–2.29 ms
reopen with 950 records to replay	68–138 ms	129–289 ms

Decision. `redb`. The fsync floor of this disk is about 7 ms per frame and both engines sit on it; where they differ, `redb` reopens twice as fast and answers the "current state of every entity" query the kernel asks constantly an order of magnitude faster, and it is one file with no background threads. `fjall`'s raw append advantage is real and would matter only once group commit removes the fsync-per-frame floor.

Divergence from Part II M1.

Planned	Actual	Why	Disposition
Simulator with virtual clock, deterministic scheduler, replay checker	A real-process crash harness: SIGKILL of a worker, tail tearing, byte-for-byte verification	The store's exit test needs real fsync and real kill semantics, which a virtual clock cannot give; the deterministic kernel simulator belongs with the kernel it simulates	Kernel simulator moves to M2; the crash harness stays as the store's standing test
Group commit on a short interval	One fsync per frame	One writer today; group commit needs concurrent appenders to batch, which arrive with executions in M2	M2
CI on aarch64	x86_64 only	Not needed yet; noted so it is not forgotten	M2 or later

Planned	Actual	Why	Disposition
Record types for Execution, Action, Completion, Compilation, Judgment as structs	Kinds reserved; structs not yet defined	They are the kernel's (M2) types; reserving the kind numbers now avoids a migration	M2
"The arena at this stage is a HashMap"	No arena at all; reads go index → WAL	Nothing needs a resident graph yet; latency is already sub-3 μs per get	M2/M5
Disk-full handling	<code>max_total_bytes</code> cap on the WAL, tested	A real ENOSPC test needs a small filesystem; the cap exercises the same code path	Real ENOSPC in the kernel simulator

Known gaps carried forward. Positions are read one index lookup at a time in `scan` (fine at this size; a sequential frame walk is the obvious optimization). `latest_of_kind` decodes every latest record; a count-only path exists. The WAL never compacts or archives segments (append-only is the model; tiering to S3 is M4 Boundaries).

A2. M2 Kernel (closed 2026-09-26, theseus-2p3)

What exists. A `theseus-kernel` crate, synchronous and deterministic (time from an injected `clock`; the only process state is the set of turn locks held), driving the M1 store through the `Store` trait, and wired into the daemon so that every live turn is a kernel turn.

- **Executions** (§3.15) as durable records keyed by id: `queued` | `running` | `waiting` | `blocked` | `cancelled` | `failed` | `budget_exhausted` | `complete`, with an `Authority` (principal, delegation, ceilings intersected on fork and never widened), a `Budget` (limit, spent, reserved, held-unknown, control reserve, open reservations), a `Wake` (`due_at`, `actions`, `execution`, `confirm`, `input`), `outstanding` and `queued_results` correlation ids, turn and interruption counters. One execution per session; the core's `SessionRecord` carries its id and sessions written before M2 get one on their next turn.
- **Actions** (§3.16) keyed by harness-minted correlation id: `planned` → `authorized` → `dispatched` → `succeeded` | `failed` | `outcome_unknown` | `cancelled`, each transition its own frame committed before the next step (the `dispatched` frame is the transactional outbox). Every action declares a **retry class** (`safe_to_repeat`, `idempotent_with_key`, `recoverable_by_external_id`, `non_repeatable`), a deadline, the sha256 digest of its canonical proposal, and an optional **confirmation binding** (confirm id, principal, bound digest, expiry). `outcome_unknown` is resolvable: later evidence settles it and is ledgered as a resolution.
- **The Completion envelope:** one type from every producer (`inproc`, `provider:<name>`, `wrapper:<pid>`, `reconciler`, `sim`). Accepting one is **idempotent and atomic with continuation**: the completion record, the settled action, the execution's next state (result queued; woken if it waited on this action), and the ledger row are one frame. A duplicate is a logged no-op; a completion for no action is quarantined under its own key and counted in health; a late completion after cancel is recorded as a resolution and revives nothing.

- **The gate** (§3.17 ordering): `transform` → `validate` → `policy` → `confirm` bound to the final digest → final revalidation at `authorize` → `dispatch`. The kernel enforces the ordering by digest: any change to the proposal after planning or after the confirm was bound fails `authorize` with `ConfirmInvalidated`; an expired or wrong-principal confirm likewise. Policies are a trait with a permissive default; M2 tests the structure, not a policy.
- **Budgets as hard limits.** A reservation is taken in the `planned` frame; a known result converts it to spent; unknown usage moves it to `held_unknown` and it is never released; a later resolution moves held to spent. Exhaustion is terminal and is written in the same frame as the refusal, so nothing keeps spending against a dry budget. A control reserve is kept back so a cancel and a final report are always affordable.
- **Admission and the turn lock.** `admit` takes a turn only for a `queued` execution, only under the ceiling (`[kernel].admission_ceiling`), only once per execution per process; it writes `running`. `end_turn` records the Advancer's decision (`complete`, `wait(wake)`, `requeue`, `fail`, `blocked`); a terminal state that landed during the turn (cancel, exhaustion) is never overwritten; ending with actions still outstanding requests their cancellation in the same frame. Waiting for admission in the daemon is a notify-and-retry, not a poll, and appears in the turn trace as `admission.wait`.
- `/cancel` acts directly on the execution and marks every dispatched action `cancel_requested`, returning the list the harness must terminate; the daemon kills wrapper process groups by pid from the spool and walks `acknowledged` → `terminated_verified` | `unsupported` | `outcome_uncertain`. Never routed through admission or a judge.
- **Promotion** (§3.2a): `promote(parent)` forks a task execution with derived authority (widening refused), a budget carved from the parent's available units, `reports_to` the parent, and the `derived_from` edge written in the same frame under the child session's scope; a parent waiting on `Wake::Execution` wakes when the child ends.
- **The spool** (`<state_dir>/spool`): completions as `tmp` → `fsync` → `rename` files, pid files, captured results, a `malformed/` quarantine; `drain` returns them oldest first and the kernel removes each file only after its frame committed, so a crash between the two redelivers a duplicate, which is a no-op.
- **The job wrapper** (`theseusd job-wrapper`): the daemon spawns itself detached in a new session (`setsid`), the wrapper runs the command, enforces its own deadline, captures output to the spool, writes the completion durably, then pokes the harness over `spool/notify.sock`. `WrapperEvidence` answers the reconciler from the spool and pid files (`completed` / `still running` / `gone`).
- **The harness loop** (`core::harness`): parked on `select()` over the heartbeat timer, the notify socket, and shutdown; store work runs off the reactor. **The reconciler** wakes due executions, checks only *overdue* dispatched actions against evidence (settling or marking unknown), re-probes unknowns, and writes one `reconcile` ledger row when anything changed.
- **Five-step startup**, each idempotent and ledgered with its duration: store recovered → executions found `running` requeued as interrupted → spool drained → reconciled → accepting. Events are refused before step 5. A test hook lets the simulator kill the process after any step.
- **The live turn.** The provider call inside every turn is now an action: `provider.messages` is planned (reservation = `max_output_tokens` + input estimate), authorized through the gate, dispatched, then settled with real token usage; an interrupted call settles as `outcome_unknown`

with its reservation held. New trace spans `admission.wait`, `action.outbox`, `action.settle`; new ledger kinds `execution.*`, `action.*`, `completion.*`, `budget.*`, `startup.step`, `reconcile`.

- **Store changes for the kernel.** Records gained a **scope** (record layout v2; manifest format 2, format-1 stores from the one day they existed are moved aside with a warning) and the index a per-scope position table, so a session's own records are one range scan (§4.4b). **Group commit** in the WAL: concurrent appenders write back to back under a short lock and one of them syncs for everyone whose bytes are written; counters for frames and syncs ride in `stats`.
- **Surface.** Protocol: `execution.list`, `execution.cancel`, `health.kernel` (accepting, ceiling, turns held, counts by state, quarantined, last startup report), `session.*.execution_state`. CLI: `theseus executions [list|cancel <id>]`; `theseus health` prints a kernel line. Config: `[kernel]` with six keys, all in the tested template. `theseusd` logs each startup step's microseconds and the store's frame/sync counts.

How it is proven.

Scenario tests (18, `theseus-kernel/src/tests.rs`, real `walstore` + `redb` in a temp dir, virtual clock): full lifecycle in one frame per transition; duplicate no-op and stray quarantine; crash mid-turn requeues as interrupted and a second startup is a no-op; completion during restart lands in the spool and startup drains it, a second copy is a duplicate; **crash inside each of the five startup steps** recovers on the next startup with the requeue counted once; cancel of a dispatched job then a late completion does not revive; unknown then genuine success resolves and moves held budget to spent; overdue action with spooled evidence settles from the reconciler and a not-yet-overdue one is not polled; a due wake fires only at its time; confirm binds the final action and any change, wrong principal, or expiry invalidates; budget is a hard limit and exhaustion is terminal; admission ceiling holds and `/cancel` is honored immediately; promotion derives authority, carves budget, writes the edge, wakes the parent; a wait on already-settled actions does not park forever; a hundred sessions mid-turn survive an upgrade (new kernel over the same store) and every one takes its next turn.

The deterministic kernel simulator (`theseus-sim kernel-sim`): virtual clock, real store and spool in a temp dir, a fake tool whose jobs finish after a scripted delay, never, twice, or after cancel; seeded fault injection of a crash **between any two kernel frames**, a crash inside a random startup step on 30 % of restarts, lost notifies (spool only), duplicate deliveries, lost jobs, and `/cancel`s; the kernel invariants checked after every step (running count equals turns held and is under the ceiling; waiting has a wake and queued has none; budget sums; every outstanding id is an unsettled action of that execution; no duplicate queued results; terminal with outstanding only as cancel-in-flight; dispatched actions are outstanding; quarantine holds only unminted ids), and at the end every durable completion settled its action or was recorded as late, every lost job ended unknown or cancelled, nothing left dispatched, nothing held.

Run	Result
40 seeds × 400 steps, redb	298 crashes (140 inside startup steps), 2 348 turns, 2 225 actions, 1 447 completions (203 duplicates, 335 notifies lost, 164 lost jobs, 72 late after cancel), 389 cancels, 197 unknown → 115 resolved, 16 040 invariant checks, all held , 45 s

Run	Result
1 seed × 2 000 steps, 30 sessions, ceiling 4, p(crash) 0.05	58 crashes, 190 turns, 174 actions, all invariants held
5 seeds × 400 steps, fjjall	26 crashes, all invariants held
1 seed × 300 steps, fsync on	all invariants held

The simulator found two real bugs before any test did: a turn ending `complete` with earlier actions still outstanding leaked them (now a cancel request in the same frame), and a completion for an action whose execution had already reached a terminal state settled the action but left it listed as outstanding (now always removed; only wake and result-queueing are skipped for terminal executions).

Group commit (`theseus-sim bench --writers N, redb, fsync on`): 1 writer 146 frames/s with one sync per frame (unchanged from M1); 4 writers 304 frames/s, one sync per 2 frames; 16 writers 1 019 frames/s, one sync per 7.6 frames; 16 writers with group commit off, 146 frames/s.

End to end (`scripts/smoke.sh`, real API): a turn produces `execution.running`, `action.planned/authorized/dispatched/succeeded`, `execution.waiting`; `theseus executions` shows the execution with its budget spent; a stray wrapper completion (`theseusd job-wrapper ... -- /bin/echo`) is quarantined and counted, never inferred into anything; the stdio server runs on its own store and spool; 67 workspace tests, clippy clean, licences clean.

Divergence from Part II M2.

Planned	Actual	Why	Disposition
Two completion transports (in-process channel, Unix-socket spool) plus SQS and loopback HTTP stubs	In-process (the provider call and the tests) and the spool with a Unix-socket notify; no SQS or HTTP stubs	Nothing would call a stub yet; the <code>Completion</code> type and <code>accept_completion</code> are the interface they will share	M4 Boundaries
Job wrapper as a systemd scope (or double-fork on the desktop)	<code>setsid</code> in a spawned child of the same binary, killed by process group	Same lifetime independence without a systemd dependency; a scope is a later hardening on hosts that have systemd	M4
Turn lock per channel with explicit release points at offload boundaries and a first durability job in released time	Turn lock per execution (§3.2a supersedes the per-channel wording); release points exist (the turn ends <code>waiting</code> on a dispatched action) but no durability job runs in released time yet	The checkpoint still happens every N records and at shutdown; the tender scheduler that uses released time is M5	M5 Tenders
Admission scheduler with budget and provider-rate-limit inputs	A concurrency ceiling only, FIFO by who asks	Rate-limit signals exist on the provider response but nothing feeds them back yet	M3/M4

Planned	Actual	Why	Disposition
Deterministic policy gate "without hooks yet"	As planned; policies are a trait, hooks do not participate in the kernel gate yet	The core's hook sites and the kernel gate meet when tools arrive	M3
reports_to delivery as ordered channel actions	reports_to recorded; no channel exists to deliver into	M4	
Sessions as compiler scopes with the per-session position table	Table built and used by session_tail; the compiler does not exist	M3 Context	
Kernel simulator: fake Discord, fake provider, fake Jev, scenario files	A fake tool, seeded random operations, in-code invariants; no scenario files	The kernel has no Discord, provider, or Jev to fake; scenario files arrive with those pieces	M3+
Group commit "on a short interval"	Leader-follower: the first appender past the write syncs for all bytes written so far; no timer	A timer adds latency for a single writer; the leader pattern gives one sync per frame alone and batching under load with no tuning	Keep
CI on aarch64 (carried from M1)	Not done	Still not needed	Later
Real ENOSPC in the simulator (carried from M1)	Not done; the WAL cap is still the disk-full path	Needs a small filesystem in CI	Later

Known gaps carried forward. The web UI shows nothing of executions or actions yet (the CLI and the ledger do). No tool other than the provider call goes through the kernel because there are no tools yet (M3). Budget units are tokens; dollars arrive with the model catalog (theseus-5xn).

Execution.authority.ceiling is a string compared for equality; the partial order for shell classes comes with the shell mapping. The notify socket is unauthenticated beyond directory permissions, which is the spec's stance for on-node transports.